

KnitWeb: A Knowledge-Graph Peer-to-Peer Weave (No Blocks, No Hashgraph)

2026 reconciliation note. This whitepaper is from the original KnitWeb companion series. In the current **knitweb/pulse** implementation the terminology has settled: the value unit is **PLS** (formerly framed as “AGT”), minted *only* as a demand-gated reward for **Proof-of-Useful-Work** and epoch-bound to the **Pulse** heartbeat — the weave/Web layer itself still introduces no coin; PLS is minted at the work/settlement layer. The peer-to-peer mesh is the **knitweb mesh**; settlement is the integer **Braid / Knit / Fiber** ledger; reasoning is **Lens**; governance is **vBank**; Sybil-resistance is **personhood**. Canonical vocabulary: *Web · Knit · Pulse · Fiber · knitweb*.

Author: E. Hauwert, VirtualV Holding B.V. **Date:** 2026-06-16 **Version:** v0.1 **Part of the original KnitWeb companion series.** In the current **knitweb/pulse** implementation the value unit is **PLS** — see the reconciliation note below.

Abstract

A peer-to-peer knowledge commons needs a data structure to hold the graph that its participants collaboratively fill and share. This paper argues that the right structure is a **weave** — a two-dimensional woven fabric of append-only, content-addressed strands — and explicitly **not** a blockchain (W1, W11) and **not** a hashgraph (W2). KnitWeb is the fabric underneath KnitNet (#4): **warp threads** are stable axes (entities, topics, schemas, authorities), **weft picks** are provenance-bearing assertions woven across them, and **interlacements** are the content-addressed crossings that are the unit of structural sharing and dedup (W4). The fabric is a state-based CRDT: peers reconcile by monotone union plus content-address dedup of strands, converging on identical structure with **no leader, no quorum, and no virtual voting** (W3). Order is total *within* a strand and partial *across* strands (W10, W12) — by design, because knowledge is multi-authored, partially ordered, and contradiction-tolerant. Crucially, structural convergence is not semantic agreement: contradictory picks coexist, each carrying PROV-O

provenance (W5) and a `[0,1]` credence, weighed by SHACL validation (W6) and KnitNet reputation/dispute (W8), never forked or voted away. Value and finality stay on MeshLattice's block-lattice (#3); the weave is for knowledge.

1. Introduction

A peer-to-peer **knowledge commons** — many independent agents collaboratively asserting, sourcing, contradicting, and refining facts — is not a ledger of value. Yet the dominant decentralised data structures were all designed to be ledgers of value, and they carry that design's assumptions with them. Before choosing one for knowledge, it is worth asking what knowledge actually *is* as a data structure, and whether those assumptions fit.

1.1 A knowledge commons needs a data structure

KnitNet (#4) defines an economy of Knowledge Assets and proof-of-knowledge, but an economy needs a substrate: the actual graph that participants fill and share, and the rules by which two peers who have never met reconcile their copies of it. That substrate is what this paper specifies. It must let any agent append an assertion at any time, let those assertions reference shared entities and schemas, dedup identical contributions made independently, and let two peers exchange and merge sub-graphs and provably end up holding the same thing — all without a coordinator. The substrate is a *structure-and-merge* problem, distinct from KnitNet's *value-and-trust* problem layered on top of it.

1.2 The thesis: a weave, not a chain or an event-DAG

A **blockchain** (W1) imposes a single global total order with objective finality; a **block-lattice** (W11) relaxes this to one total-ordered chain per account, still with finality. A **hashgraph** (W2) builds a global total order and deterministic finality over an event-DAG via gossip-about-gossip and virtual voting. All three are, at heart, machines for manufacturing **agreement on order** so that value cannot fork. That is exactly the right primitive for money and exactly the wrong primitive for knowledge.

We argue the right structure for a knowledge commons is a **weave**: a two-dimensional fabric of append-only, content-addressed strands (W4, W10), woven from two kinds of thread.

- **Warp threads** are the stable axes — persistent entities, topics, schemas, and authorities — the long-lived, content-addressed identifiers across which knowledge is laid.

- **Weft picks** are the assertions woven *across* the warps: each pick is a provenance-bearing statement (an RDF triple/quad, W7) linking warps, carrying PROV-O provenance (W5) and a `[0,1]` credence. A KnitNet Knowledge Asset (#4) is a bundle of weft picks.
- **Interlacements (knots)** are the content-addressed points where a weft crosses a warp — the atomic unit of structural sharing and dedup, deterministically named so two peers independently mint the same knot.

The whole fabric is the union of all threads and interlacements; that union **is** the knowledge graph. Because every primitive is keyed by a CID over its canonical bytes, two peers reconcile by **set union plus content-address dedup** — the fabric is a join-semilattice and a state-based CRDT (W3): merge is commutative, associative, idempotent, and monotone. Peers therefore converge on byte-identical structure **with no leader, no quorum, and no virtual voting** — the deliberate difference from chains (W1, W11) and hashgraph (W2).

1.3 Knowledge is partially ordered and contradiction-tolerant

The mismatch between value-ledgers and knowledge is not incidental; it is structural.

- **Knowledge is multi-authored and partially ordered.** Two agents asserting unrelated facts at the same time are not competing for a slot in a global sequence; their statements are simply *incomparable*. Causal order matters *within* a line of derivation — and the weave preserves it via per-strand append-only links and Lamport happens-before (W10, W12) — but forcing all facts into one canonical history is invented coordination that knowledge does not need and that acts as a centralising pressure.
- **Knowledge is contradiction-tolerant.** A double-spend must be rejected; a *disputed fact* must not be. Two sources that disagree are a normal, information-bearing state of a knowledge graph, not an error to be resolved by picking a winning branch. The weave therefore lets contradictory weft picks **coexist**, each with its own provenance and credence.

This forces a sharp distinction the rest of the paper develops: **structural convergence is not semantic agreement**. Everyone eventually holding the same fabric does not mean they agree on what is true. Conflicts are **weighed, not erased** — by credence, by SHACL validation-by-replay (W6), and by KnitNet's stake-plus-reputation dispute mechanism (time-decayed EigenTrust, W8) — and never by forking the fabric or voting a fact away. Sybil and abuse resistance comes from KnitNet (#4), not from the weave structure: **KnitWeb adds no new token and no new consensus**. Where an application genuinely needs total order or objective finality — token transfers, irreversible commitments — it uses MeshLattice's block-lattice (#3).

Value and finality stay on the ledger; knowledge lives on the weave; the two compose over the same knitweb mesh transport and PLS/DID identity (#1, W9).

1.4 Contributions

This paper makes the following contributions.

- **The WEAVE data model.** A formal two-dimensional fabric of append-only, content-addressed strands — warp threads, weft picks, and interlacements — as the data structure underneath KnitNet (#4), with canonical schemas for each primitive (§2.1–§2.2).
- **Convergence without consensus.** A proof-shaped treatment of the fabric as a state-based CRDT (W3): merge by CID-keyed set union and dedup is commutative, associative, idempotent, and monotone, yielding byte-identical convergence with no leader, quorum, or virtual voting (§2.3).
- **An explicit order model.** Total order *within* a strand (append-only, hash-linked, Lamport-clocked; W10, W12) and partial order *across* strands — and a precise statement of what KnitWeb deliberately does **not** provide: no total order over the fabric, no objective finality on facts, no consensus-on-order (§2.3).
- **Truth ≠ structure.** A model in which contradictory picks coexist with PROV-O provenance (W5) and `[0,1]` credence, weighed by SHACL validation (W6) and KnitNet reputation/dispute (W8) rather than resolved by the weave (§2.4).
- **A three-way comparison** of blockchain (W1, W11), hashgraph (W2), and KnitWeb across order, finality, multi-author support, contradiction-tolerance, consensus mechanism, and what each sacrifices (§2.5).
- **A clean separation of concerns:** knowledge on the KnitWeb weave, value and finality on MeshLattice's block-lattice (#3), composing over a shared knitweb mesh transport and PLS/DID identity (#1, W9) without either layer imposing its ordering on the other (§2.6).

2. Background

This section reviews the data structures and ordering primitives that an open, multi-author knowledge graph might be built on, and what each one *provides* versus what it *costs* when the goal is collaboratively filling and sharing a knowledge graph rather than moving value. The contrast frames the KnitWeb design decision (Section 3 onward): knowledge is partially ordered, multi-authored, and contradiction-tolerant, so the primitives that buy a *single canonical history* are buying the wrong thing.

2.1 Blockchains: a single global total order with finality

A blockchain in the Nakamoto sense (W1) is a **1-D chain of blocks**: each block hash-links to its predecessor, and a consensus rule (proof-of-work in W1, proof-of-stake and others since) selects, among competing forks, one canonical chain. The output is a **global total order** over all committed transactions plus **objective finality**: once a transaction is buried sufficiently deep, all honest participants treat it as irreversible, and a contradicting transaction (a double-spend) is by construction *invalid* — exactly one branch survives and the other is discarded.

- **Provides.** A single agreed history; no-double-spend; tamper-evidence via hash-linking; a definite "this happened, in this position" answer for every transaction.
- **Costs for a knowledge graph.** The defining strengths are liabilities for knowledge. Writers are **serialized** into one history, which is a centralizing pressure and a throughput bottleneck. Forks must be *resolved away* — but contradiction between two assertions is the normal, desirable state of a knowledge commons, not an error to be pruned. There is **no native multi-author concurrency** and **no contradiction-tolerance**: the structure cannot represent "A says X, B says $\neg X$, both stand with provenance and credence." Imposing a total order on facts answers a question — *which fact came first?* — that knowledge curation rarely needs.

2.2 Block-lattice: per-account total order

Nano's block-lattice (W11) refines the chain into **one chain per account**: each account owns and extends its own ledger, and a transfer is a send-block on the sender's chain matched by a receive-block on the receiver's chain. Ordering becomes **per-account total** rather than globally total, removing the single-writer bottleneck; cross-account agreement (which send pairs with which receive, and which of two conflicting blocks on the *same* account wins) is reached by Open Representative Voting — a stake-weighted vote among delegated representatives (W11).

- **Provides.** Parallel, per-writer append (each account writes its own chain independently); still **objective finality** on transfers, via the representative vote that picks a winner whenever an account double-signs.
- **Costs for a knowledge graph.** The block-lattice shows that *per-writer* append-only chains compose well — a structural idea KnitWeb reuses at the strand level. But it still resolves conflicts by **voting one block away** to keep each account's chain a single line with finality. For a knowledge graph that is the wrong primitive twice over: assertions are not owned by one account-chain, and conflicting assertions must **coexist**, not be voted

out. The lattice keeps the value-ledger requirement (one winner, finality) that knowledge does not want.

2.3 Hashgraph: total order via gossip-about-gossip and virtual voting

Hashgraph (W2) reaches consensus without an explicit chain. Each participant records *events*, every event hash-links to two parents — the participant's own previous event and the last event received from a peer — and these events spread by **gossip-about-gossip**: peers exchange not just transactions but the history of *who told whom, and when*. Propagation is **randomized pairwise gossip** — each node periodically syncs with a random peer — so the gossip-about-gossip history spreads epidemically and reaches all nodes in $O(\log n)$ rounds, not by any all-to-all message pattern. From this gossip history every node can locally reconstruct what every other node would have seen and run **virtual voting** — computing the votes other nodes *would* have cast without sending any vote messages. The result is a **total order over all events** with deterministic, Byzantine-fault-tolerant **finality** (W2).

- **Provides.** A globally consistent total order and fast deterministic finality on every event, with low per-decision message overhead (votes are inferred, not transmitted).
- **Costs for a knowledge graph.** Hashgraph still delivers the very thing a knowledge commons should *not* want: a single ordered history with finality on every event. It is a **1-D-per-participant event-DAG** whose purpose is to collapse to one sequence. The cost is not fan-out — KnitWeb gossips pairwise too — but the **metadata and voting machinery** that the total order demands: every event must carry two parent hashes plus the gossip-about-gossip history needed to run virtual voting, overhead spent purely to manufacture order and finality. And, as with blockchains, it offers no representation for contradictory assertions coexisting with provenance and credence. Ordering and finality are computed at cost, then thrown at a problem (knowledge) that is partially ordered by nature.

2.4 CRDTs and join-semilattices: convergence without consensus

Conflict-free Replicated Data Types (W3) provide a different guarantee entirely. In a **state-based CRDT** (CvRDT), each replica holds a state drawn from a **join-semilattice**, and replicas reconcile by repeatedly applying the lattice **join** (least-upper-bound, $\sqcup$) of their states. The join is **commutative, associative, and idempotent**, and on a join-semilattice these properties characterize the least-upper-bound: replicas that have observed the same set of updates converge to **identical state regardless of message order, duplication, or delay** — with **no leader, no quorum, and no voting** (W3). **Monotonicity follows**: on a join-semilattice the join is

inflationary ($F \sqsubseteq F' \sqsubseteq F''$ for all states), so merging only ever moves a replica *up* the lattice — state grows, never silently shrinks. It is not an extra axiom layered on top of the three; it is a consequence of the lattice structure. (In KnitWeb this is the grow-only, no-deletion property: retraction is modelled as a further assertion, not as removal.) Crucially, CRDTs deliver **convergence of structure** while deliberately *not* establishing any total order over the updates.

- **Provides.** Eventual, deterministic convergence under arbitrary network behaviour; native multi-writer concurrency; no consensus protocol, no finality machinery.
- **Costs / limits for a knowledge graph.** A CRDT guarantees that replicas hold the *same* state — it does **not** adjudicate which assertions are *true*. Structural convergence is necessary but not sufficient: meaning, contradiction, and trust must be layered on top (in KnitWeb: PROV-O provenance, credence, SHACL validation, and KnitNet curation). This is precisely the right trade for knowledge — pay nothing for ordering you do not need, and handle truth as a separate, explicit concern.

2.5 Content-addressing and Merkle DAGs

Content-addressing identifies an object by a cryptographic hash of its bytes rather than by location or name. IPFS/IPLD (W4) generalize this into a **Merkle DAG**: objects reference one another by hash, so any object's identifier transitively commits to its entire dependency closure. Two consequences matter here. First, **identity is content** — equal content yields one identifier, so independently produced copies of the same object **deduplicate** automatically. Second, references are **tamper-evident**: changing any byte changes the hash and breaks every link that pointed at the old version.

- **Provides.** Global dedup, structural sharing of common sub-structure, verifiable integrity, and cheap diff/reconcile via Merkle roots (compare roots; descend only where they differ).
- **Use for a knowledge graph.** Content-addressing is the *key space* that makes CRDT merge (Section 2.4) trivially correct: if elements are keyed by a content identifier, set union performs dedup for free and the join is exact. It supplies the addressing layer; it does not by itself say how strands are ordered or how truth is weighed.

2.6 Append-only feeds: per-author signed logs

Secure Scuttlebutt (W10) models each identity as a single **append-only, signed, hash-linked feed**: every message carries the hash of the previous message on that author's feed and is signed by that author's key. This yields a **per-author total order** that is cryptographically

verifiable — any gap, reordering, or edit is detectable, because it breaks the hash chain or the signature.

- **Provides.** Tamper-evident, attributable, totally-ordered *per author*; partial replication (a peer can hold some feeds and not others) and offline-first operation.
- **Use for a knowledge graph.** Append-only feeds give the **per-strand** building block: a single author's strand is a verifiable, totally-ordered log. They say nothing about ordering *across* authors — which is exactly the freedom a multi-author knowledge graph needs, and which CRDT merge then reconciles structurally without imposing a global sequence.

2.7 Causal order and logical clocks

Lamport (W12) established that distributed events admit a **partial order** — *happens-before* — defined by causality: within one process events are ordered by their sequence, and a send happens-before its matching receive; events on different processes with no such causal path are **concurrent** and **incomparable**. Logical (Lamport) clocks assign each event a counter consistent with this partial order, and the partial order can be *extended* to a total order if an application needs one — but doing so requires an arbitrary tie-breaking rule and discards the information that the events were genuinely concurrent.

- **Provides.** A principled, message-driven notion of order — total *within* a process, partial *across* processes — with cheap logical timestamps and no global clock.
- **Use for a knowledge graph.** Lamport's distinction is the conceptual backbone of the design: **within** a strand, causality gives a true total order (matching the append-only feed of Section 2.6); **across** strands, concurrent assertions are incomparable and should *stay* that way. Blockchains, block-lattice, and hashgraph all spend effort to extend this partial order into a total one with finality. A knowledge graph should **not** pay that cost: the partial order is not a deficiency to be eliminated but the correct model of a multi-authored, contradiction-tolerant commons.

2.8 Summary: what each primitive provides and costs for a knowledge graph

Primitive	Order it gives	Finality	Multi-author	Contradiction-tolerant	Net fit for a knowledge graph
Blockchain (W1)	Global total	Objective	Serialized to one history	No (forks resolved away)	Wrong primitive: buys a single canonical history knowledge does not want
Block-lattice (W11)	Per-account total	Objective (via ORV vote)	Per-account chains	No (conflicts voted away)	Reusable idea (per-writer chains); still votes conflicts out
Hashgraph (W2)	Global total	Objective, deterministic BFT	One ordered history	No	Wrong primitive: total order + finality via gossip-about-gossip metadata + virtual voting
CRDT / join-semilattice (W3)	None imposed (convergent state)	None	Native	Yes (at structural level)	Core merge primitive: convergence without consensus
Content-addressing / Merkle DAG (W4)	n/a (addressing)	n/a	n/a	n/a	Key space enabling exact dedup + verifiable diff
Append-only feed (W10)	Per-author total	n/a	Per-author logs	n/a	Per-strand building block: verifiable, attributable log

Primitive	Order it gives	Finality	Multi-author	Contradiction-tolerant	Net fit for a knowledge graph
Causal / Lamport order (W12)	Within-process total; across-process partial	n/a	n/a	n/a	The right order model: total within strand, partial across

The pattern is consistent: blockchains, the block-lattice, and hashgraph each invest substantial machinery to manufacture a **single totally-ordered history with finality** (W1, W11, W2) — the correct primitive for *value*, where double-spends must be impossible and exactly one history must win. A knowledge graph has the opposite shape: it is multi-authored, partially ordered, and tolerant of standing contradiction. The remaining ingredients — CRDT join over a content-addressed key space (W3, W4), per-author append-only feeds (W10), and Lamport's within/across distinction (W12) — compose into convergence *without* consensus on order, leaving truth to be weighed explicitly rather than decided by the structure. That composition is the KnitWeb weave.

This is a self-contained academic authoring task. The content is fully specified in the brief and reference list, so I'll write the section directly per the canonical spec.

3. Why Neither a Block nor a Hashgraph for Knowledge

This section argues, from the nature of knowledge itself, that the two canonical decentralized-ordering primitives — the blockchain (total order + objective finality, W1, W11) and the hashgraph (total order + finality via gossip-about-gossip and virtual voting, W2) — are the **wrong primitives for a knowledge commons**. The argument is not that these mechanisms are flawed; on their own terrain — *value* — they are exactly right, and KnitWeb delegates that terrain to them (§2.6, #3). The claim is narrower and structural: knowledge is **partially ordered, multi-authored, and contradiction-tolerant**, and imposing a single canonical history or global finality on facts is at best unnecessary overhead and at worst an actively centralizing, knowledge-destroying constraint.

3.1 What a knowledge commons actually requires

A knowledge graph is not a ledger of transactions; it is an open, growing body of **assertions** authored concurrently by many parties who need not agree. Three properties are intrinsic to it, and each is in direct tension with consensus-on-order.

Property of knowledge	What it means	What an ordering consensus does to it
Partially ordered	Most assertions are <i>causally independent</i> : "slag batch H2-2026 has pH 9.1" and "EigenTrust is time-decayed" stand in no happens-before relation. Only derivation (<code>prov:wasDerivedFrom</code>) and within-source revision are genuinely ordered.	A blockchain (W1) or hashgraph (W2) computes a total order over all events , manufacturing a sequence between facts that have no real ordering relation — pure overhead with no semantic content.
Multi-authored	Many peers assert about the same warp simultaneously, in good faith, without coordination. There is no single writer and no canonical author of "the truth about" an entity.	Both primitives serialize writers into one canonical history ; concurrent assertions are forced through a single ordered timeline, and "who got there first" becomes load-bearing where it should be irrelevant.
Contradiction-tolerant	Two well-sourced assertions can disagree ("pH 9.1" vs. "pH 8.7"), and the commons must hold both with provenance and credence until evidence, replay, or dispute resolves them (§2.4). Disagreement is information, not corruption.	A total-order ledger admits one history : a fork or double-write is <i>invalid</i> , one branch wins and the other is erased. Contradiction is modelled as fault, so the commons cannot represent its most common and useful state — open disagreement.

A consensus protocol exists to answer one question: *in what single order did these events happen, and which is final?* For value that question is essential (§3.4). For knowledge it is **mostly meaningless** — there is no single order — and where it is meaningful (within a source, by derivation) it is already captured *locally* by the append-only, hash-linked strand (§2.1, W10, W12) without any global agreement.

3.2 Against the blockchain: total order + finality on facts is the wrong primitive

A blockchain (W1) — and equally a block-lattice such as Nano's per-account chains with representative-voted finality (W11) — provides a **globally (or per-account) total order with objective finality**. Three consequences make this a poor fit for a knowledge commons.

1. **Manufactured order is overhead, not information.** To place independent assertions into one chain (W1) or to settle each account's chain head (W11), the protocol must run leader election, proof-of-work/stake, or representative voting. That machinery's entire output is *an order* — precisely the artefact a knowledge graph does not need across threads (§3.1). The cost (block production, voting rounds, confirmation latency) is paid to produce a serialization that carries no semantic value for facts.
2. **Objective finality on facts is undesirable.** Finality means *irreversible*. But a knowledge claim should **never** be structurally irreversible: new measurement, a failed SHACL replay (W6), or a successful dispute (#4) must be able to contradict, supersede, or re-weight it at any time (§2.3.3, §2.4). A primitive whose defining feature is "this is now permanent and one-history-true" is in direct conflict with the epistemics of a commons that is meant to *revise*. KnitWeb instead retracts via a `prov:wasInvalidatedBy` weft pick — monotone addition, never deletion (§2.3.1).
3. **One canonical history is a centralizing pressure.** A single agreed history requires the network to converge on *who decides the order*: miners, stakers, or elected representatives (W1, W11). Whoever controls ordering controls which assertions enter and in what sequence — a chokepoint and a gatekeeper. An open, contradiction-tolerant commons must let any peer weave a pick and let conflicts coexist; routing every fact through an ordering authority reintroduces exactly the central control the suite is built to avoid.

In short, the blockchain optimizes for the property knowledge least needs (a single irreversible order) and pays for it with the properties knowledge most needs (openness, partial order, contradiction-tolerance).

3.3 Against the hashgraph: virtual voting still produces the wrong artefact

The hashgraph (W2) is more efficient and more elegant than chain-based consensus: peers exchange events via **gossip-about-gossip**, and each peer computes a **total order and finality by virtual voting** — inferring how others would have voted from the shared event-DAG, with no actual vote messages. It is an excellent consensus algorithm. But for a knowledge commons it is the right machine aimed at the wrong target.

- **The output is still a single total order with finality** — exactly the artefact §3.1–§3.2 showed is undesirable for facts. A more efficient route to the wrong primitive is still the wrong primitive. Virtual voting buys cheap *agreement on order*; a knowledge commons does not want agreement on order, it wants **coexistence with provenance**.
- **It assumes a single shared history is the goal**. Hashgraph's correctness rests on every participant converging to one ordered, final sequence of events (W2). Knowledge has no such canonical sequence to converge to; the assumption that finality is even desirable does not hold for revisable, multi-authored assertions (§3.1).
- **Gossip-about-gossip is a 1-D-per-account event-DAG**, not a woven fabric. It orders *events*; KnitWeb shares *structure*. Two KnitWeb peers do not need to agree on the order in which picks were gossiped — only to hold the **same set** of content-addressed strands, which set-union + CID dedup achieves directly (§2.3.1, W3, W4) with no DAG of events, no famous-witness computation, and no virtual election.

The contrast is sharpest here: hashgraph and KnitWeb both avoid heavyweight leaders and both achieve deterministic convergence, but hashgraph converges on an **order of events** (a total sequence) while KnitWeb converges on a **state** (a set of facts). For knowledge, the state is the goal and the order is an artefact.

3.4 Where order *is* needed — and where it lives

Rigour requires the converse claim: there are real needs for total order and objective finality, and KnitWeb does not pretend otherwise. **Value** is the canonical case. A token transfer must have a single agreed history and must be irreversible — double-spends are prevented precisely by a total order with finality, which is what a blockchain (W1) or a block-lattice with representative voting (W11) delivers. An irreversible commitment (a stake, a payment, a signed obligation) is meaningless without finality.

KnitWeb's design response is **separation, not denial** (§2.6): where an application needs total order or objective finality it uses **MeshLattice's block-lattice (#3, W11)**; where it needs an open, contradiction-tolerant, partially-ordered knowledge commons it uses the **KnitWeb weave**. The two compose over the same knitweb mesh transport and PLS/DID identity (§2.6), and may cross-reference by CID — a pick may cite a ledger transaction as `prov:wasDerivedFrom`, a ledger commitment may anchor a fabric root manifest — but **neither imposes its ordering on the other**. The ledger never totally-orders knowledge; the weave never grants value objective finality.

3.5 Summary: matching the primitive to the data

The error both primitives would make, applied to knowledge, is a **type error**: using an *order-and-finality* mechanism on data that is *partially ordered and revisable*.

	Blockchain / block-lattice (W1, W11)	Hashgraph (W2)	KnitWeb weave (§2)
Computes	Global / per-account total order + finality	Total order + finality via virtual voting	Convergent state (set of facts)
Order across independent facts	Manufactured (overhead)	Manufactured (overhead)	None — partial order only, by design
Reversibility of a fact	None (final, one history)	None (final, one history)	Always revisable (supersede / invalidate / re-weight)
Concurrent multi-author assertions	Serialized into one history	Serialized into one history	Coexist with credence + provenance
Centralizing pressure	Ordering authority (miners/stakers/repes)	Convergence on one canonical sequence	None — any peer weaves; conflicts coexist
Correct domain	Value / finality (its right job)	Value / finality (its right job)	Knowledge commons (its right job)

The conclusion is not that consensus-on-order is bad — it is indispensable for value, and the suite keeps it for exactly that on the block-lattice (#3). It is that **knowledge is the wrong workload for it**. A knowledge commons needs convergence of *structure*, not agreement on *order*; KnitWeb obtains that convergence from content-addressing and CRDT merge (W3, W4) — no leader, no quorum, no virtual voting — and leaves total order and finality to the layer whose data genuinely demands them.

4. The Weave

This section instantiates the formal model of §2 (Canonical Design Spec) as the operative description of the KnitWeb fabric. It defines each primitive — warp threads, weft picks, interlacements, fabric/net, and selvedge — shows how a KnitNet Knowledge Asset (#4) maps onto a set of weft picks woven across warps, and closes with a text diagram of a woven fabric.

Throughout, every primitive is identified by a **CID** (Content IDentifier): a multihash over its canonicalized bytes, so that identity *is* content and byte-equal objects deduplicate to a single node (W4).

4.1 Primitives of the weave

KnitWeb is a **2-D woven fabric** rather than a 1-D chain or a 1-D-per-account event-DAG. The fabric is built from five primitives, each a content-addressed object or a set thereof.

Primitive	Symbol	Role in the weave
Warp thread	$w \in W$	A stable <i>axis</i> — a persistent entity, topic, schema, or authority — across which assertions are woven. The "vertical" thread of the fabric; long-lived and content-addressed.
Weft thread (pick)	$p \in P$	A provenance-bearing <i>assertion</i> woven across one or more warps: an RDF triple/quad (W7) carrying PROV-O provenance (W5) and a $[0,1]$ credence. The "horizontal" thread. Its structural core is an edge (the bare statement); credence/provenance/asset are envelope fields.
Edge (statement)	$e \in E$	The bare RDF statement $\{s,p,o,g?\}$ at a pick's core, content-addressed over canonical $\{s,p,o,g?\}$ and nothing else (matching #4 §5.1). The structural unit of dedup and credence fusion : identical statements from different authors coalesce to one cid_e .
Interlacement (knot)	$k \in K$	The content-addressed <i>point</i> where an edge e crosses a warp w in a given role; a deterministic adjacency node derived from cid_e , not from any per-author envelope.
Fabric / net	$F = (W, P, K)$	The union of all warps, wefts (with their edges), and interlacements — this <i>is</i> the knowledge graph.
Selvedge	$s \in S$	The verified <i>edge/index</i> of a sub-weave: a signed, content-addressed index over a coherent region of F , carrying its validation status and frontier.

4.1.1 Warp threads — the stable axes

A **warp thread** $w \in W$ is the long-lived axis of the weave: a persistent entity (e.g. "Acme Steel Slag, batch H2-2026"), a topic, a SHACL schema, or an authority. A warp is not a single record

but an **append-only strand of version records** — renames, schema revisions, and authority key-rotations are *appended* (each carrying `supersedes` to the prior version), never mutated in place. The warp's stable identity is `cid_w`, the CID of its genesis record; every later record on the strand carries `prev = cid` of the record before it (§2.2.1–2.2.2). Because the warp is the thing across which many independent authors weave assertions, it must remain stable and shareable: content-addressing makes two peers that mint the same entity converge on the same `cid_w` without coordination (W4).

4.1.2 Weft threads (picks) — assertions across warps

A **weft thread**, or **pick**, $p \in P$ is a single provenance-bearing assertion woven *across* warps. Structurally a pick is an **envelope** around a bare RDF statement (its **edge**, §4.1.3): the edge `{subject, predicate, object, graph?}` references warps (or canonical IRIs/literals), and the envelope bundles:

- a `[0,1]` **credence** — the author's subjective confidence in *this* assertion;
- **PROV-O provenance** (W5): `wasAttributedTo` (the signing DID/PLS, #1, W9), `wasDerivedFrom` (source picks or external resources), `wasGeneratedBy` (e.g. replay-validation, measurement, LLM-extraction), and `generatedAtTime` ;
- the `asset` bundle CID, if any (§4.2).

A pick is itself an append on a weft strand, hash-linked by `prev` and signed (§2.2.1, §2.2.3). The pick CID covers the full envelope (so two authors making the *same* statement mint *different* pick CIDs — they are distinct, separately-attributed assertions), but the pick **carries** its edge CID `cid_e`, computed over the canonical statement only. This is the crucial separation, inherited verbatim from KnitNet §5.1: **the assertion is the edge**, and structural identity lives at the edge, not the envelope. Crucially, a weft is an *assertion*, not a fact decreed true: contradictory picks over the same warps **coexist** in `F`, each weighed by its own credence and provenance rather than resolved by the structure (§2.4).

4.1.3 Edges and interlacements — content-addressed structure

The structural core of a pick is its **edge** $e \in E$: the bare statement `{s, p, o, g?}`, content-addressed over its canonical encoding (§5.1) **and nothing else** — credence, provenance, author, and asset are *excluded* from the preimage:

```
cid_e = CID( canonicalize({ subject, predicate, object, graph? }) )
```

This rule is taken verbatim from KnitNet (#4 §5.1, W14), and it is what makes cross-author and cross-asset dedup work: **the same statement asserted by two authors, or carried in two Knowledge Assets, yields the identical** `cid_e`, so the edge is held once and the two authors' credences fuse over that shared edge per KnitNet §5.3 — rather than splitting into two unrelated nodes. (Had we content-addressed the structural unit by the pick CID — which folds in credence/provenance/author — this dedup and fusion would silently fail, since no two independently-authored picks would ever coincide.)

An **interlacement** (knot) `k ∈ K` is then the point where an **edge** `e` crosses a warp `w` in a specific `role ∈ {subject, predicate, object}` (a fixed enum). Its identity is derived deterministically from the **edge** CID, using the *same* canonical, domain-separated, length-delimited encoding as every other preimage in this paper (§5.1) — never bare byte-concatenation:

```
cid_k = CID( canonicalize({ "knot": 1, "edge": cid_e, "warp": cid_w, "role": role })
```

Two peers that independently observe the *same* crossing of the *same* statement therefore mint the *identical* `cid_k`, with no concatenation-ambiguity. Note that `cid_k` is **fully determined by** `cid_e` (the edge already commits to its subject/predicate/object warps and their roles): the knot adds no new convergence guarantee that edge-level union does not already give. It earns its keep solely as a **materialized adjacency index** — `K` is the set of content-addressed (`warp ↔ edge, role`) crossings, letting a peer enumerate "every statement touching warp `w`" by a CID lookup rather than a scan of `P`. Where that index is not needed, knots may be omitted and recomputed on demand from `E`; they are derivative, not load-bearing for convergence.

4.1.4 The fabric / net — the knowledge graph

The **fabric** `F = (W, P, K)` is the componentwise union of all warp threads, weft threads (each carrying its edge `e`), and interlacements held by a peer. This union *is* the knowledge graph: warps are its nodes-of-record, edges its labelled edges, picks the provenance-bearing assertions of those edges, and knots the shared crossings that bind edges to warps. A fabric is summarized by a **root manifest** — a CID over the set of strand-head CIDs (a Merkle structure, W4) — which lets two peers diff and reconcile their fabrics cheaply without transferring the whole net. Because membership is by CID and merge is set union, every peer that has exchanged the same strands converges on a **byte-identical fabric** (§2.3.1), and identical edges coalesce regardless of how many distinct picks assert them.

4.1.5 Selvedge — the verified edge of a sub-weave

The **selvedge** $s \in S$ is the finished, verified edge of a sub-weave — the index by which a region of the fabric is offered to and verified by peers. A selvedge is a signed, content-addressed object over `{covered strand-heads, version-vector frontier, SHACL-validation report (W6), signer DID (W9)}` (§2.1). It lets a peer **verify and request a coherent sub-weave** — one authority's warps, or one Knowledge Asset's picks — and confirm its validation status and frontier *without holding all of* F . The selvedge is thus both the integrity boundary and the sync handle of a sub-weave.

4.2 A Knowledge Asset as a set of weft picks across warps

A **KnitNet Knowledge Asset** (#4, W14) is not a new primitive in the weave: it is a **named bundle of weft picks woven across shared warps**. Where KnitNet treats a Knowledge Asset as the unit of proof-of-knowledge, curation, and dispute, KnitWeb treats that same asset as a *region of the fabric* — a coherent set of picks $\{p_1, \dots, p_n\} \subseteq P$, each tagged with the asset's CID via the pick's `asset` field (§2.2.3), each carrying an edge in $E_A \subseteq E$, all crossing a common set of warps $W_A \subseteq W$ and minting the knots $K_A \subseteq K$ at those crossings.

Concretely, an asset materializes as:

Asset element	Weave realization
Asset identity	<code>cid:bafyKnowledgeAsset</code> — the bundle CID referenced by every member pick's <code>asset</code> field.
Asset claims	A set of weft picks $\{p_1 \dots p_n\}$, each enveloping an edge <code>cid_e</code> (an RDF statement over warps in W_A , W7) with its own credence + PROV-O provenance (W5).
Asset entities/topics	The warps W_A the picks' edges cross (entities, topics, schemas, authorities).
Asset structure	The interlacements K_A : the deterministic knots where the asset's edges cross its warps (§4.1.3).
Asset boundary/index	A selvedge s_A over $\{W_A, P_A, K_A\}$ carrying the SHACL-validation report and frontier — the verifiable handle to fetch and check the asset (§4.1.5).

Two consequences follow. First, **sharing is automatic at the edge level**: if two assets both assert the *same statement* about the warp "Acme Steel Slag, batch H2-2026", that warp and the

shared **edge** (and its knots) are held once and shared between the two asset-regions by content-address (§4.1.3) — assets overlap on the fabric rather than copying each other, and the two authors' credences fuse over the one shared edge (#4 §5.3). The two picks remain distinct (each keeps its own provenance and credence envelope), but the structure they assert is deduplicated. Second, **assets compose without merging their meaning**: distributing or merging an asset is a CRDT union of its strands (§2.3.1), so an asset can be replicated, partially fetched (via its selvedge), or woven into a larger fabric with no re-ordering and no consensus step — while any contradiction between two assets' picks is preserved and weighed by credence/provenance/dispute (§2.4), not silently resolved.

```
// A Knowledge Asset (#4) = a bundle of weft picks (P_A) over shared warps (W_A),
// indexed by a selvedge (s_A). Each pick ENVELOPES an edge (cid_e) computed over
// the bare statement {s,p,o,g?} ONLY — credence/prov/asset are envelope, excluded
// from cid_e — so identical statements dedup + fuse across authors/assets (#4 §5.1,
{
  "asset": "cid:bafyKnowledgeAsset",          // bundle identity (#4)
  "selvedge": "cid:bafySelvedge_sA",          // verified edge/index over this region
  "warps": ["cid:bafyAcmeSlagH2", "cid:bafyTRL4", "cid:bafyAssayX"], // W_A
  "picks": [                                   // P_A — each is one weft pick (§2.2.3)
    { "statement": { "subject": "cid:bafyAcmeSlagH2", "predicate": "cid:bafyHasTRL", "
      "edge": "cid:bafyEdgeHasTRL",           // cid_e = CID(canonicalize({s,p,o}))
      "credence": 0.82, "asset": "cid:bafyKnowledgeAsset" }, // envelope-only, NOT
    { "statement": { "subject": "cid:bafyAcmeSlagH2", "predicate": "cid:bafyMeasuredBy
      "edge": "cid:bafyEdgeMeasuredBy",
      "credence": 0.91, "asset": "cid:bafyKnowledgeAsset" }
  ]
}
```

4.3 Loss-free MeTTa projection (interoperable with #4)

A fabric region projects to MeTTa/Hyperon atoms (W13) using the **same atom shapes as KnitNet §5.2** (#4, W14), so a single pattern-match works identically over both papers' output. Each **edge** becomes a function-style atom (predicate subject object), and metadata are atoms keyed by the **edge CID** — *not* the pick CID:

```

; edge cid_e = bafyEdgeHasTRL projects to the SAME atom KnitNet (#4 §5.2) emits:
(kn:hasTRL kn:entity/AcmeSlagH2 kn:entity/TRL4)

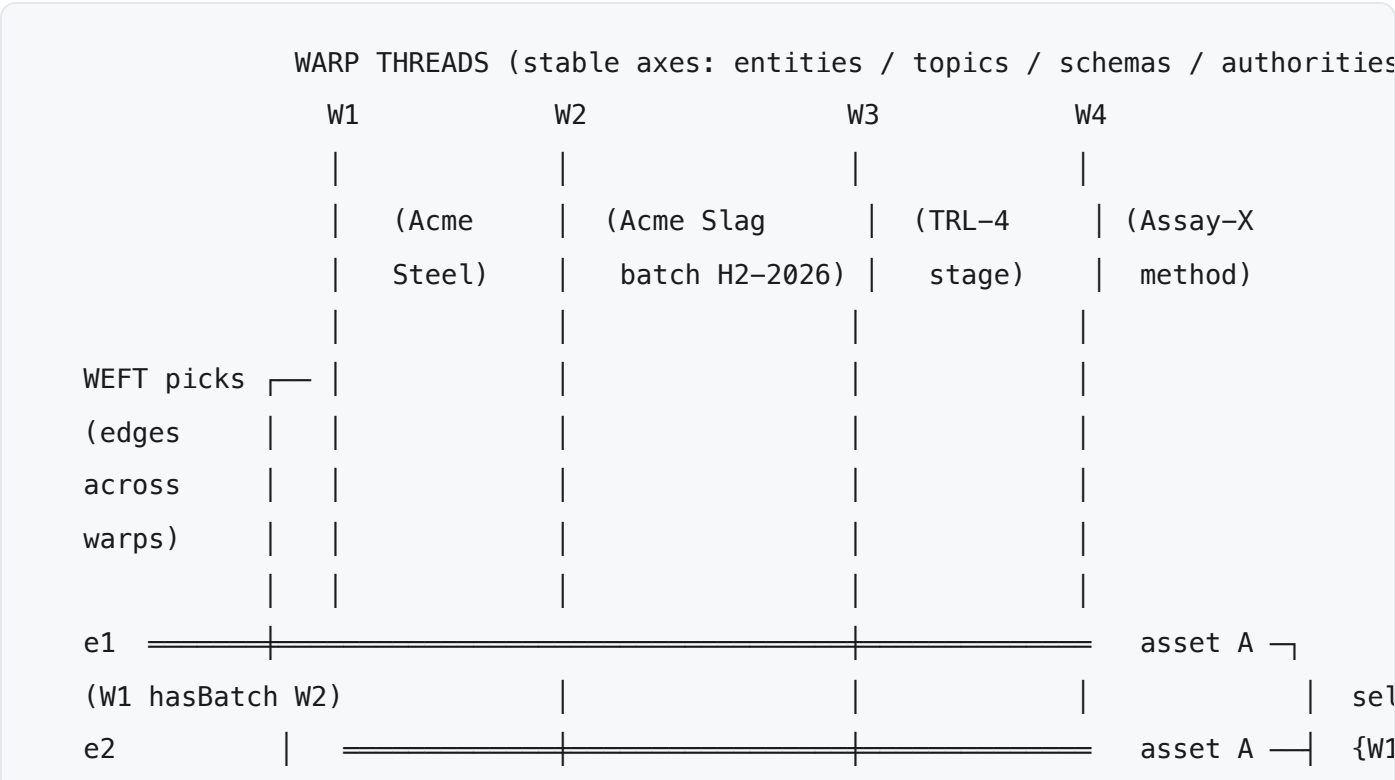
; metadata keyed to the EDGE CID (matching #4 §5.2 verbatim), so credences fuse per
(credence (edge bafyEdgeHasTRL) 0.82)
(provenance (edge bafyEdgeHasTRL) (wasAttributedTo did:agt:z6MkAuthorA) (wasGenerated
(credence (edge bafyEdgeHasTRL) 0.79) ; a SECOND author's pick over the SAME edge

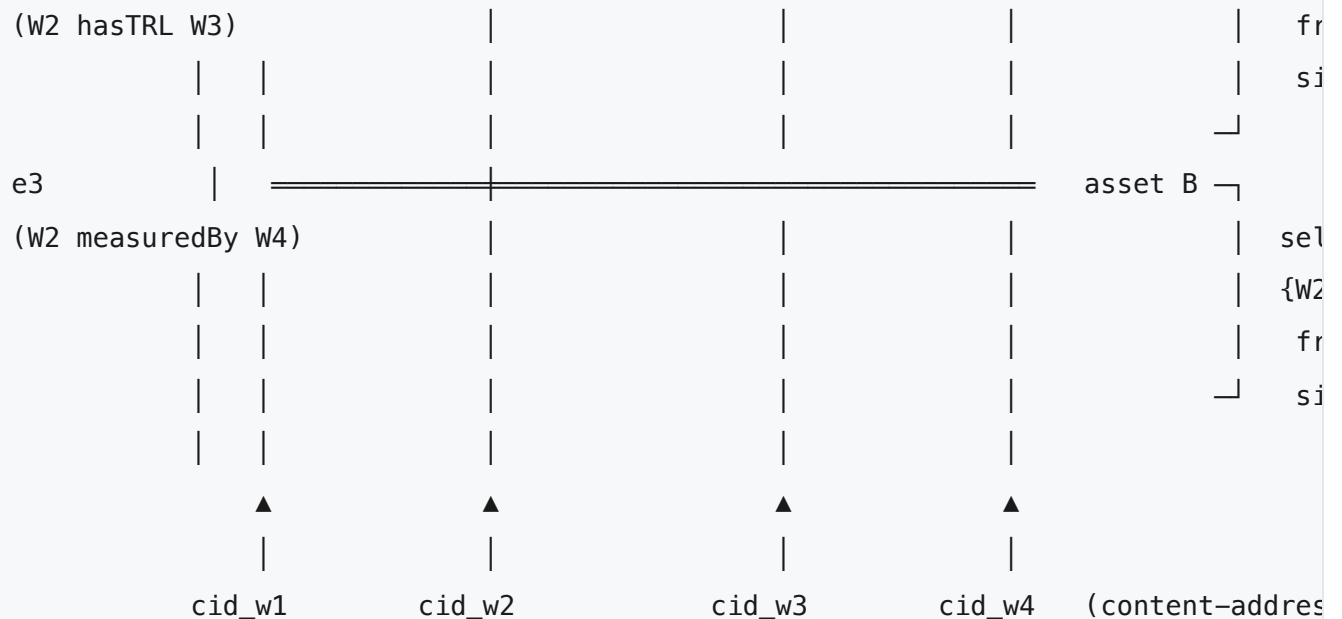
```

Because the structural atom and its metadata keys are addressed by `cid_e`, two independently-authored picks of the same statement project to **one** (predicate subject object) atom with **two** credence atoms fusing over it — exactly the cross-author behaviour #4 §5.3 specifies, and byte-for-byte the same atoms a KnitNet projection of the same edge would emit. KnitWeb adds no divergent reification: the pick's envelope (author, `prev`, signature) is recoverable from the strand but is never the atom key.

4.4 A text diagram of the fabric

The figure below shows warps as **vertical** strands (`|`), weft-pick edges as **horizontal** threads (`=`) woven across them, and interlacements as the crossing knots (`⊥`). Two Knowledge Assets share a common warp (`w2`), illustrating structural sharing; the right margin shows each asset's selvedge as the finished, indexed edge of its sub-weave.





$\neq$ = interlacement (knot): $\text{cid}_k = \text{CID}(\text{canonicalize}(\{\text{knot}, \text{edge:cid}_e, \text{warp:cid}_w, \text{r} \dots$
 – derived from the EDGE cid_e (not the pick), determin
 = = weft-pick edge: RDF statement $\{s, p, o, g?\} \rightarrow \text{cid}_e$ over canonical $\{s, p, c \dots$
 the pick envelopes it with credence $[0, 1] + \text{PROV-0}$ pro
 | = warp thread (axis): append-only, hash-linked, content-addressed strand

Note: warp W2 is SHARED by assets A and B – held once by content-address (W4), woven into both sub-weaves rather than copied. If two assets asserted the SAME statement, its EDGE (cid_e) would also be held once and the two author credences would FUSE over it (#4 §5.3). Contradictory picks over the same w simply coexist as additional = threads, each with its own credence and prov (§2.4) – the fabric grows, it is never forked.

In this fabric, the **union of all** |, =, and $\neq$ is the knowledge graph $F = (W, P, K)$; each weft-pick edge is content-addressed by cid_e over its bare statement, so identical statements coalesce and fuse across authors and assets (#4 §5.1, §5.3); each Knowledge Asset (#4, W14) is the bounded sub-weave to the right of its picks, made fetchable and verifiable by its **selvedge**; and the whole structure is reconciled between peers by content-addressed union (§2.3), with no global block, no per-account chain, and no consensus on order.

5. Data Structure and Encoding

This section instantiates the primitives of §2 as concrete, content-addressed wire/storage objects, defines their append-only causal links, gives a loss-free projection of a weft pick into both RDF (W7) and OpenCog Hyperon MeTTa (W13), and shows how deduplication falls out of content-addressing (W4). All objects are **canonicalized before hashing** (deterministic key order; canonical RDF term encoding per W7), so byte-equality $\Rightarrow$ CID-equality $\Rightarrow$ a single stored node.

5.1 Content-addressing and canonical encoding

Every primitive is named by a **CID** (Content IDentifier): a self-describing multihash over the canonicalized bytes of the object (W4). The CID *is* the identity of the object; there is no separate, mutable key. Two consequences follow directly and are exploited throughout KnitWeb:

- **Identity = content.** Any peer that holds the same bytes computes the same CID without coordination. A reference (`prev` , `strand` , `wasDerivedFrom` , `subject/object warp`) is itself a CID, so the fabric is a Merkle structure (W4) in which every link is verifiable by re-hashing the target.
- **Equal content deduplicates.** Because the CID is derived from canonical bytes, two independently authored objects with identical content collapse to one node under merge (see §5.5). This is the structural basis of the CRDT union of §2.3.

```
canonicalize(obj) → bytes          // deterministic key order, canonical RDF terms (
cid(obj)          = multihash(canonicalize(obj)) // self-describing hash (W4)
```

The `cid` field of a record is computed over the canonical bytes of **all other fields** and appended last; verifiers strip `cid` , re-canonicalize, re-hash, and compare.

5.2 Thread strand record (append-only, hash-linked)

A **strand** is an append-only, hash-linked, signed log (W10) — used for both warp threads (stable axes) and weft threads (assertion picks). Each record carries `prev` (the CID of the prior record on *this* strand), a per-strand monotonic `seq` , and a Lamport clock (W12), giving every strand a verifiable **total internal order** (§2.3.2).

```
{
  "type": "strand_record",          // append-only log entry (W10)
  "strand": "cid:bafyWarp0rWeftHead", // stable strand identity = CID of its genesis
```

```

"kind": "warp" | "weft",           // axis (warp) or assertion-pick (weft)
"seq": 42,                         // per-strand monotonic sequence (total interr
"prev": "cid:bafyPrevRecord|null", // hash-link to prior record on THIS strand; r
"lamport": 1187,                   // Lamport clock; total-order-COMPATIBLE times
"author": "did:agt:z6Mk...",       // signing identity (W9 / #1 PLS)
"payload": { /* warp_meta | weft_pick – see §5.3, §5.4 */ },
"sig": "ed25519:...",             // signature over canonical bytes of this reco
"cid": "cid:bafyThisRecord"        // CID of the canonicalized record (self-id, s
}

```

Causal-link invariants.

- `seq = 0` $\Leftrightarrow$ `prev = null` (genesis). For `seq > 0`, `prev` MUST reference the record with `seq - 1` on the same `strand`; this is the hash-chain that makes tampering detectable — any edit changes the record's `cid`, breaking the successor's `prev`.
- `strand` equals the `cid` of the genesis (`seq = 0`) record and never changes; it is the strand's stable identity.
- `lamport` is non-decreasing along a strand and, on each append, is set past the maximum Lamport value of any record referenced via cross-strand `wasDerivedFrom` (§5.4) — the standard Lamport update rule (W12).

Two distinct ordering mechanisms — keep them apart. The design carries the actual **happens-before** relation in explicit links, and uses Lamport values only as a tie-break-compatible timestamp; the two must not be conflated:

- **Happens-before (the real causal order) is carried by explicit edges.** Within a strand it is the `prev` $\rightarrow$ `prev` $\rightarrow$... $\rightarrow$ `null` hash-chain; across strands it is the explicit `prov:wasDerivedFrom` edges (§5.4). The transitive closure of these two edge sets *is* the partial order over the fabric (§2.3.2): record `a` happens-before record `b` iff a directed path of `prev` / `wasDerivedFrom` edges runs from `b` back to `a`. Two records with **no** such path in either direction are **concurrent (incomparable)** — and this is decided by the *absence of a causal path*, not by any comparison of timestamps. Concurrency/incomparability reported by the query engine (§6.2, §8.4) is therefore read off these explicit edges.
- **Lamport clocks provide only a total-order-compatible timestamp.** They satisfy the one-way property `a` $\rightarrow$ `b` $\Rightarrow$ `L(a) < L(b)` (W12), which makes them safe for *optional* deterministic tie-breaking when a caller wants a stable linear presentation of otherwise-incomparable picks. The converse does **not** hold: `L(a) < L(b)` does **not** imply `a` $\rightarrow$ `b`, so

comparing two Lamport values can **never** establish that picks are concurrent or causally related. (Deciding concurrency from timestamps alone would require vector clocks; KnitWeb deliberately uses explicit causal edges instead, which is sufficient and cheaper to store.)

- No order is implied between records on different strands beyond this explicit-edge partial order (§2.3.2); Lamport values impose no causal claim of their own.

5.3 Warp payload (stable axis)

A warp strand's payload describes a persistent entity, topic, schema, or authority. Versioning (renames, schema revisions, authority key-rotation) is expressed as a new append whose payload `supersedes` the prior version's CID — never an in-place mutation.

```
{
  "warp_id": "cid:bafyWarpGenesis",    // == strand identity; immutable
  "class": "entity" | "topic" | "schema" | "authority",
  "label": "Acme Steel Slag, batch H2-2026",
  "schema": "cid:bafyShaclShape|null", // SHACL shape governing picks on this warp (W)
  "supersedes": "cid:bafyPriorVersion|null" // version chain for renames / key-rotation
}
```

5.4 Weft pick payload (provenance-bearing assertion)

A weft pick is a single RDF triple/quad (W7) woven across warps, carrying PROV-O provenance (W5), a $[0,1]$ credence, and the interlacement (knot) CIDs it mints. Each `subject/predicate/object` term is either a warp CID or a canonical IRI/literal, so the assertion is fully content-addressed and re-derivable.

[illegible]

```

    "wasAttributedTo": "did:agt:z6Mk...",
    "wasDerivedFrom": ["cid:bafySourcePick", "ipfs://..."],
    "wasGeneratedBy": "activity:replay-validation|measurement|llm-extract",
    "generatedAtTime": "2026-06-16T00:00:00Z"
  },
  "asset": "cid:bafyKnowledgeAsset|null", // bundle this pick belongs to (#4)
  "interlaces": [                        // the knots this pick mints (see §5.4.1)
    "cid:bafyKnotSubject", "cid:bafyKnotObject"
  ]
}

```

The entries of `wasDerivedFrom` are themselves CIDs (of source picks) or content-addressed URLs, so provenance is a **verifiable causal back-link** in the Merkle fabric (W4): it is precisely the explicit cross-strand happens-before edge of §5.2, and it is also what advances the Lamport clock on append.

5.4.1 Interlacement (knot) record

The crossing of a weft pick over a warp at a given role is content-addressed **deterministically** from its three constituents, so two peers that independently observe the same crossing mint the *same* `cid` and the knot deduplicates to one stored node (§5.5).

```

{
  "type": "interlacement",
  "warp": "cid:bafyWarp",           // the crossed axis
  "weft": "cid:bafyWeftPick",       // the crossing assertion
  "role": "subject" | "predicate" | "object", // crossing role
  // DETERMINISTIC identity – any peer minting this crossing gets the SAME cid:
  "cid": "cid:bafyKnot = CID(warp || weft || role)"
}

```

5.5 Deduplication by content address

Because identity is derived from canonical content, deduplication is not a separate garbage-collection pass but an **invariant of naming**. It operates at three levels, each a direct consequence of §5.1:

Level	What collapses	Mechanism
Record	Two byte-identical strand records	Same canonical bytes $\Rightarrow$ same <code>cid</code> $\Rightarrow$ one node under union (W4).
Knot	The same <code>(warp, weft, role)</code> crossing minted by different peers	Deterministic <code>cid = CID(warp weft role)</code> $\Rightarrow$ identical CID without coordination (§5.4.1).
Sub-structure	Shared warps/sources referenced by many picks	References are CIDs; a warp or source pick is stored once and pointed to by every <code>subject/object / wasDerivedFrom</code> that names it (Merkle sharing, W4).

This is exactly the merge operator of §2.3: `F1  $\sqcup$  F2` is componentwise CID-keyed set union, and union over CID keys *is* dedup. Merging is therefore **idempotent** (re-importing a strand is a no-op) and **commutative** (import order is irrelevant) — convergence with no leader and no consensus on order (§2.3.1). Retraction does not delete a node (which would violate monotonicity); it is a *new* weft pick asserting `prov:wasInvalidatedBy` over the target (§2.3.1).

5.6 Loss-free RDF / MeTTa projection of a pick

A weft pick is designed to project **without loss** into (a) RDF (W7) for standard graph tooling and (b) OpenCog Hyperon MeTTa atomspace (W13) for symbolic reasoning. The mapping is total and invertible: the canonical pick is the source of truth, and both projections are recoverable views — re-canonicalizing either projection yields the same `cid`.

To stay representationally consistent with the suite, KnitWeb adopts KnitNet's chosen idiom (#4, §4.3 / §5.2): statement-level metadata is keyed to the **content-addressed edge/pick CID**, and where a reifying RDF term is needed it uses an **RDF 1.2 triple-term referenced by** `rdf:reifies` — *not* the older verbose RDF reification idiom and *not* an asserting- `<<...>>` RDF-star quotation, which #4 explicitly supersedes. The pick CID is the single content-addressed handle that both projections hang metadata from, so the two papers share one representation.

The pick (canonical fields):

```
{
  "statement": { "subject": "cid:bafyAcmeSlag",
                 "predicate": "ex:hasPhosphorusFraction",
```

```

      "object": "\"0.07\"^^xsd:decimal" },
    "credence": 0.82,
    "prov": { "wasAttributedTo": "did:agt:z6MkEdwin",
              "wasGeneratedBy": "activity:replay-validation",
              "generatedAtTime": "2026-06-16T00:00:00Z" },
    "cid": "cid:bafyPick77"
  }

```

(a) RDF projection (W7), provenance keyed to the pick CID. The asserted base triple stands on its own (W7). The pick CID `<cid:bafyPick77>` is a first-class resource that carries credence + PROV-O metadata (W5); it identifies the asserted statement via an **RDF 1.2 triple-term** in object position, bound with `rdf:reifies` per KnitNet §4.3 — superseding the older reification idiom and keeping credence/provenance on the content-addressed edge exactly as #4 §5.2 does:

```

# base triple (the woven assertion over warps, W7) – asserted in the graph
<cid:bafyAcmeSlag> ex:hasPhosphorusFraction "0.07"^^xsd:decimal .

# the pick as a first-class, content-addressed edge; metadata keyed to its CID.
# rdf:reifies binds the pick CID to the RDF 1.2 triple-term it describes (KnitNet §4
# WITHOUT re-asserting it (the base triple above is the assertion).
<cid:bafyPick77> a kw:WeftPick ;
    rdf:reifies <<( <cid:bafyAcmeSlag> ex:hasPhosphorusFraction "0.07"^^xsd:decimal
    kw:credence "0.82"^^xsd:decimal ;
    prov:wasAttributedTo <did:agt:z6MkEdwin> ;
    prov:wasGeneratedBy <activity:replay-validation> ;
    prov:generatedAtTime "2026-06-16T00:00:00Z"^^xsd:dateTime .

```

(Pure edge-CID keying — dropping the `rdf:reifies` triple-term and relying on `kw:WeftPick` fields plus the `interlaces` knot CIDs to recover the statement — is the leaner variant #4 §5.2 calls "cleaner still"; the `rdf:reifies` term above is retained only where downstream tooling wants the statement spelled out as an RDF term, and it is the RDF 1.2 form, never `<<...>>` - asserting RDF-star.)

(b) MeTTa projection (W13), preserving the same structure as atoms in the atomspace. The pick CID is the reifying handle; warp CIDs, credence and provenance are carried as nested

expressions so no field is dropped:

```
; the woven assertion, named by the pick CID (content address, W4)
(= (pick cid:bafyPick77)
  (Statement (Subject   cid:bafyAcmeSlag)
              (Predicate ex:hasPhosphorusFraction)
              (Object    (Literal "0.07" xsd:decimal)))))

; credence and PROV-O provenance (W5) keyed to the same pick CID
(credence cid:bafyPick77 0.82)
(prov cid:bafyPick77
  (wasAttributedTo did:agt:z6MkEdwin)
  (wasGeneratedBy  activity:replay-validation)
  (generatedAtTime "2026-06-16T00:00:00Z"))
```

Round-trip guarantee. Both projections preserve (i) the warp-CID terms of the statement, (ii) the pick's own CID as the key to which metadata attaches, (iii) credence, and (iv) PROV-O provenance (W5). Because metadata is keyed to the content-addressed pick CID in both views (and any RDF reifying term is the RDF 1.2 `rdf:reifies` form, matching #4), re-canonicalizing the fields recovered from either projection reproduces the identical byte string and therefore the identical `cid` (§5.1) — so a pick may round-trip $\text{RDF} \rightleftharpoons \text{canonical} \rightleftharpoons \text{MeTTa}$ with no information loss and no change of identity.

6. Convergence Without Consensus

KnitWeb converges by *replication*, not by *agreement*. Two peers that have observed the same set of strands hold the same fabric — but no peer, quorum, or algorithm ever decides a canonical order of facts. This section gives the formal model from the Canonical Design Spec (§2.3): the fabric as a join-semilattice, reconciliation by union with content-address dedup, the order guarantees that follow, and an explicit statement of the guarantees KnitWeb deliberately withholds — with the argument for why their absence is *correct* for a knowledge commons rather than a missing feature.

6.1 The fabric as a state-based CRDT

A peer's local state is the fabric $F = (W, P, K)$: three sets of CID-keyed objects — warp threads, weft picks, and interlacements (§2.1). Reconciliation of two peers' states is defined as componentwise set union:

$$F_1 \sqcup F_2 = (W_1 \cup W_2, P_1 \cup P_2, K_1 \cup K_2)$$

Because every element is keyed by a **CID** — a multihash over its canonical bytes (W4) — set union performs **automatic deduplication**: equal content yields an equal CID and therefore one element, regardless of how many peers independently minted it. The pair $(\mathfrak{F}, \sqcup)$ is a **join-semilattice**, and the fabric is a **state-based CRDT** (CvRDT) in the sense of Shapiro et al. (2011) (W3). The merge operation has the three algebraic properties that make replicas converge:

Property	Statement	Why it holds
Commutative	$F_1 \sqcup F_2 = F_2 \sqcup F_1$	Set union is order-independent; merge order does not matter.
Associative	$(F_1 \sqcup F_2) \sqcup F_3 = F_1 \sqcup (F_2 \sqcup F_3)$	Set union is associative; grouping of merges does not matter.
Idempotent	$F \sqcup F = F$	Re-merging an already-held strand is a no-op; duplicates collapse on CID.

To these the model adds **monotonicity**: merging only ever grows the fabric,

$$F \sqsubseteq F \sqcup F' \quad \text{where} \quad F_1 \sqsubseteq F_2 \iff F_1 \subseteq F_2 \text{ componentwise,}$$

so knowledge is **added, never silently removed**. Retraction is not deletion: to withdraw an assertion an author appends a *new* weft pick carrying a `prov:wasInvalidatedBy` link (W5), which is itself monotone state. The fabric is therefore a grow-only structure whose lattice order $\sqsubseteq$ is exactly subset inclusion.

Dedup is at the edge/knot level, not the pick level. A weft *pick* carries per-author, per-time envelope fields — `author`, `sig`, `seq`, `prev`, `lamport`, `credence`, `prov`, `generatedAtTime` (§5.2, §5.4) — so two authors who independently assert the *same* statement mint **distinct** pick CIDs, and their picks remain distinct strands in P . This is correct: each assertion is its own provenance-bearing act. Structural sharing happens one level down, at the **interlacement**

(knot), whose preimage is taken over the *edge/statement* CID — the canonical (subject, predicate, object, graph) tuple of warp CIDs (the KnitNet unit, §5.1) — **not** over the pick CID:

```
cid_k = CID( cid_w || cid_edge || role )      where  cid_edge = CID(s, p, o, g)
```

Because `cid_edge` is independent of which author asserted it or when, two peers observing the same crossing of the same statement over the same warp **mint the identical** `cid_k`, and the knot deduplicates across authors on union. Thus the fabric shares structure at the edge/knot layer while preserving every independent assertion as a separate, attributable pick — the "unit of structural sharing and dedup" claim holds precisely at the knot, and only there.

Convergence theorem (informal). Commutativity, associativity and idempotency are precisely the conditions under which a state-based CRDT converges (Shapiro et al., 2011) (W3). Consequently any two peers **that have observed the same set of strands** hold **byte-identical fabrics**, irrespective of message order, duplication, or delay. This is the standard CRDT guarantee — *strong eventual consistency*: replicas that have delivered the **same set of updates** converge to equal state (Shapiro et al., 2011) (W3). The guarantee is therefore **conditional on the shared update set**, not unconditional.

Partial replication and shared closure. This caveat is load-bearing, because partial replication is a headline feature: via **selvedge** sub-weaves (§4.1.5, §8.2) a peer routinely holds only a *region* of the fabric. Two peers with different subscriptions hold different **sub-fabrics** and are *not* byte-identical overall. What CRDT convergence guarantees is convergence **on their shared closure**: for the set of strands both peers have observed, the merge is identical down to the byte; the regions only one peer holds simply differ until exchanged. Because merge is monotone and order-independent, every strand a peer *later* acquires merges in with no conflict and no rework — so partial replicas are always consistent on their overlap and grow monotonically toward fuller agreement as their observed sets converge. The CID Merkle structure makes this reconciliation cheap as well as correct: a peer compares **root manifests** (a CID over its set of strand-head CIDs), descends only where they differ, and requests exactly the missing strands — diff-and-pull over a woven Merkle fabric (W4), never a replay of a global log.

6.2 Order guarantees: total within a thread, partial across threads

Convergence on *state* is distinct from agreement on *order*. KnitWeb provides order at exactly one scale and refuses it at the other.

Within a thread — total order. Every strand (a warp or a weft thread) is an append-only, hash-linked log (§2.1): record `n` carries `prev = cid(n-1)`, a per-strand monotone sequence `seq`, and a Lamport timestamp (W10, W12). The hash-link makes the sequence **cryptographically verifiable and tamper-evident** — any edit to record `n` changes its CID, breaking the `prev` pointer of record `n+1` and every record after it. Records on a single warp or weft are therefore **totally ordered by construction**, and that order is self-authenticating.

Across threads — partial order only. Between distinct strands there is no global clock and no imposed sequence. The only order that holds is the **causal partial order** induced by Lamport happens-before (W12) together with explicit `prov:wasDerivedFrom` provenance links (W5):

- if pick `q` cites pick `p` via `wasDerivedFrom`, then `p` causally precedes `q`;
- Lamport clocks give each record a logical timestamp consistent with this happens-before relation (Lamport, 1978) (W12);
- two picks from different authors with no derivation path between them are **concurrent** — incomparable in $\sqsubseteq$ — and KnitWeb places them in **no** global sequence.

The frontier summarizes replication progress, not cross-strand causality. A peer also maintains a **per-strand frontier**: the set of per-strand `seq` heads it currently holds, keyed by strand CID. This is a *replication-progress summary* — "how far along each thread the peer has reconciled" — and it is the basis for computing sync deltas (which records another peer is missing). It is deliberately **not** called a version vector: a classic version vector maps actor/replica $\rightarrow$ counter and supports causal comparison (dominance/concurrency) *between* replicas, whereas the per-strand frontier records only per-log progress and **does not by itself encode cross-strand causal dependencies**. That cross-strand causality lives elsewhere — in the **Lamport clocks** on each record and the explicit `prov:wasDerivedFrom` edges (W12, W5) — and the frontier must not be treated as a causality oracle. The frontier tells a peer *what to fetch*; the Lamport clocks and derivation edges tell it *what causally precedes what*. Concurrent picks coexist as siblings in the partial order, exactly as concurrent events do in Lamport's model (W12).

6.3 What KnitWeb does NOT provide — and why that is correct

The following are absent **by design**, not by omission. Each is a property a value ledger needs and a knowledge commons does not — and in several cases actively must not have.

Withheld guarantee	What it would mean	Why KnitWeb omits it
Total order over the fabric	A single global sequence of all facts.	No such sequence is defined or computed. Knowledge is multi-authored and partially ordered; serializing all assertions into one line is meaningless and would impose an arbitrary, centralizing canonical history.
Objective finality on facts	A pick becomes "confirmed" / irreversible.	No pick is ever frozen by the structure. Any assertion may be contradicted, superseded, or re-weighted later; treating a fact as final is epistemically wrong and would prevent correction.
Consensus-on-order	A leader/quorum/stake-vote or gossip-about-gossip virtual voting (W2) chooses an ordering.	The CRDT converges <i>structure</i> without agreeing on <i>order</i> ; no ordering needs deciding, so no consensus mechanism is run.
Structural conflict resolution of meaning	The structure forks or votes a contradiction away.	Contradictions are first-class: opposing picks coexist, each with provenance and credence (§7 / §2.4). Meaning is <i>weighed</i> , never erased by the data structure.

Why withholding these is the right choice. A blockchain (W1) or block-lattice (W11) exists to enforce a single agreed history so that value cannot be double-spent; a hashgraph (W2) reaches deterministic finality on a total order of events through all-to-all gossip and virtual voting. Both pay for that history with a single canonical timeline — which is exactly the wrong primitive for knowledge. Knowledge is **partially ordered** (most facts are simply unrelated in time), **multi-authored** (no privileged writer serializes it), and **contradiction-tolerant** (disagreement is information to be weighed, not a fault to be resolved away). Forcing a total order or an objective finality onto such a corpus would (i) destroy concurrency by serializing independent assertions, (ii) make error-correction impossible by freezing facts, and (iii) reintroduce a centralizing pressure — whoever controls the ordering controls the canon.

KnitWeb therefore keeps only the guarantee a knowledge commons actually needs — **convergence of state under monotone CRDT merge, conditional on the shared update set (W3)** — and pushes total order and objective finality to the layer that genuinely requires them: MeshLattice's block-lattice (#3, §2.6). Structural convergence here means peers that have observed the same strands eventually hold the same *fabric* (and partial replicas agree on their

shared closure); it explicitly does **not** mean they agree on what is *true*. That separation — same data, weighed meaning — is taken up next.

I now have all the canonical facts needed. I have full grounding for all three fixes:

1. **EigenTrust**: proposed/future-work, requires pre-trusted seed + missing trust-edge source; deployed primitive is stake-weighting + economic slashing; freshness via VC expiry/snapshot currency, not "time-decay".
2. **L2 canonicalization**: contradiction nodes gate promotion to L2; HotStuff BFT seals canonical merges. L0/L1 is append-only/consensus-free.
3. **Credence fusion**: governed monotone per-edge fusion rule, recomputed deterministically by all nodes (corroboration-by-replay), bounded/capped — not purely a querier-side presentation choice.

7. Meaning, Contradiction, and Credence

This section is governed by §2.4 (*Truth ≠ Structure*) and applies it in full. The central claim is a separation that the rest of the design depends on: **structural convergence is not agreement on truth**. KnitWeb's CRDT (§2.3) guarantees that peers who have exchanged the same strands hold a byte-identical fabric $F = (W, P, K)$; it guarantees nothing about whether the assertions woven into that fabric are *correct, mutually consistent, or believed*. The weave converges the container, not its meaning.

One scope note frames the whole section. The weave described in this paper is the **raw, append-only fabric** — the L0/L1-equivalent layer of KnitNet (#4): everything any author has ever woven, retained in full. The CRDT-convergence and contradiction-coexistence claims below are claims about *that* layer. KnitNet additionally derives a **canonical (L2) view** on top of the raw fabric, and that derivation is *not* consensus-free: it gates on contradiction and is sealed by BFT finality (§7.5, §7.7). KnitWeb's "no consensus, no finality on facts" is therefore precise rather than universal — it describes how the raw fabric *accumulates and reconciles*, not how a downstream canonical view is *promoted*.

7.1 Two convergences, deliberately decoupled

It is useful to name the two notions that a naive design conflates and that KnitWeb keeps strictly apart:

- **Structural convergence** — every peer eventually holds the same set of threads and interlacements in the raw fabric. This is achieved by monotone CRDT union + content-address dedup (§2.3.1), with no leader, quorum, or vote (W3, W4).
- **Semantic agreement** — peers agree on which assertions are *true*. This is **never** produced by the raw weave, and KnitWeb does not attempt to manufacture it at the structural layer.

A blockchain or hashgraph (#3, and the contrast targets of §2.5) couples these two: a fact is "real" once and only once it is in the single canonical, totally-ordered history, and a contradicting fact is therefore *invalid by construction* — one branch wins, the other is discarded. The raw KnitWeb fabric refuses this coupling. Because knowledge is multi-authored, partially ordered, and frequently contested, forcing a single canonical history *over the raw fabric* is the wrong primitive (§2.3.3): it would either silently destroy minority assertions or require a centralizing authority to adjudicate truth at write time. The raw weave instead lets the fabric carry *all* the assertions and weighs them *after the fact*, separately, and revisably. Where the suite genuinely needs a single canonical answer it derives one as an explicit, finalized L2 view (§7.5) — it does not collapse the raw fabric to produce it.

7.2 Contradiction is first-class: contradictory picks coexist

In the raw fabric, two weft picks that assert incompatible statements about the same warps are **both** valid fabric members. They are distinct strands with distinct CIDs; union admits both; dedup collapses neither (their canonical bytes differ). Nothing in the structural layer prefers one over the other, and nothing removes the loser — because at the structural layer there is no loser.

Consider two authors making opposite claims about the same warp (an entity `w_5`, a predicate, an object warp `w_0`):

```
// Pick A — author α asserts the statement, high credence
{
  "kind": "weft",
  "statement": { "subject": "cid:bafyWarpS",
                 "predicate": "cid:bafyWarpPredicate",
                 "object": "cid:bafyWarp0" },
  "credence": 0.91,
  "prov": {
    "wasAttributedTo": "did:agt:zAlpha...",
    "wasDerivedFrom": ["cid:bafyLabReportA"],
```

```

    "wasGeneratedBy": "activity:measurement",
    "generatedAtTime": "2026-06-15T09:00:00Z"
  }
}

// Pick B – author β asserts the OPPOSITE, also with provenance
{
  "kind": "weft",
  "statement": { "subject": "cid:bafyWarpS",
                  "predicate": "cid:bafyWarpPredicate",
                  "object": "cid:bafyWarp0_other" },
  "credence": 0.74,
  "prov": {
    "wasAttributedTo": "did:agt:zBeta...",
    "wasDerivedFrom": ["cid:bafyLabReportB"],
    "wasGeneratedBy": "activity:measurement",
    "generatedAtTime": "2026-06-15T14:30:00Z"
  }
}

```

After any exchange, both peers hold **both** picks. The disagreement is preserved as data — a recorded, attributable, dated dispute between α and β — rather than erased into the winner's history. A consumer of the raw fabric sees a *contested region*, not a false certainty.

This is the design intent of **monotone growth** (§2.3.1): in the raw fabric, knowledge is added, never silently removed. Even *retraction* is itself a weft pick — a `prov:wasInvalidatedBy` assertion (W5) — and not a deletion. An author who later disbelieves Pick A weaves a new pick invalidating it; the original remains in the fabric, now visibly superseded, so the audit trail of *who believed what, when, and why* is never lost. (How KnitNet's L2 canonicalization treats such a contradiction is a *derivation* step layered on top, not a mutation of the raw strands — §7.5.)

7.3 Every pick carries provenance and credence

Coexistence is only useful because each pick is **self-describing** about its origin and its author's confidence. Two fields (§2.2.3) carry this:

- **prov — PROV-O provenance (W5).** Each pick records `wasAttributedTo` (the signing DID/PLS author), `wasDerivedFrom` (source picks, documents, or ledger transactions), `wasGeneratedBy` (the activity — measurement, replay-validation, LLM extraction), and `generatedAtTime`. Provenance is what makes a contradiction *legible*: a reader can ask *not* "which fact won" but "where did each claim come from, and do I trust that path?" Provenance proves *who said what, citing what* — it does **not** prove the cited source is genuine, independent, or pre-existing (a gap that corroboration alone cannot close; W14, KnitNet §9.3).
- **credence — a $[0,1]$ value, per attribution.** The value carried *on a pick* is the **author's own subjective confidence** in the statement, asserted at write time. At the per-attribution level it is explicitly *not* a network-wide probability of truth, *not* a vote tally, and *not* a finality score. It is one attributable datum: "α was 0.91 sure; β was 0.74 sure."

Because per-attribution credence is per-author and per-pick, it is itself contestable: a third author may weave a pick that derives from both and assigns a *lower* credence to A's claim.

Per-attribution vs. fused per-edge credence. Per-attribution credence is not the whole story. When multiple *independent* attributions assert the **same edge CID**, KnitNet (#4, W14) combines their credences under a **governed, monotone fusion rule** that every node recomputes identically from the same edge set — *corroboration-by-replay* (W14, KnitNet §6.5/§9.4). This yields a **fused per-edge credence** that is network-deterministic and fabric-derived, not a per-reader presentation choice. Two properties make the fusion safe rather than sybil-amplifiable (W14, KnitNet §9.4):

- **Bounded by claimant weight, not count.** Each attribution's marginal contribution is weighted by the claimant's **stake at risk** and reputation weight, *not* by the bare number of attributions. N zero-stake, low-reputation Sybils contribute negligibly more than one.
- **Capped, sub-certainty ceiling.** The governed rule imposes a hard cap on marginal gain per additional attribution and a ceiling below `1.0` reachable from corroboration alone; externally-anchored or demonstrably pre-existing sources are weighted strictly above attacker-generable ones.

The distinction matters for KnitWeb's claims: aggregation is **not** purely querier-side. There are two legitimate aggregates — the deterministic fused per-edge credence above (recomputed by replay, the same for all nodes) and any *further* querier-side view a consumer chooses to compute (§7.4). Neither overwrites the underlying per-attribution credences, both of which remain in **F** in full; fusion is monotone and never replaces an existing attribution.

7.4 Conflicts are weighed in the raw fabric, and gated in canonicalization

When an application or curator must *act* on a contested region — render a "current best view", answer a query, accept a Knowledge Asset — it **weighs** the coexisting picks. KnitWeb specifies three weighing inputs (§2.4), and explicitly excludes, *for the raw fabric*, the two mechanisms a ledger would use to rewrite history (forking, voting a fact away — §7.5).

Weighing input	What it contributes	Source
Credence (per-attribution + fused per-edge)	The author's <code>[0, 1]</code> self-asserted confidence on each pick, plus the network-deterministic fused per-edge aggregate recomputed by replay under the governed monotone rule.	§2.2.3, §7.3, W14 (KnitNet §6.5/§9.4)
SHACL validation-by-replay (W6)	Re-executes a pick's generating activity against the warp's SHACL shape to confirm the assertion <i>still validates</i> against its declared constraints. A pick whose replay fails is down-weighted regardless of its credence.	§2.4, W6
KnitNet dispute / curation (#4)	Explicit disputes plus stake-weighted curation from the KnitNet layer (W14). Reputation biases this weighting where available; the operative anti-Sybil guarantee today is economic (stake + slashing), not reputational (§7.6).	§2.4, W14

Validation-by-replay (W6) deserves emphasis because it is the weave's check that is *intrinsic to the data* rather than to the social layer. Each warp may declare a SHACL shape governing the picks woven onto it (§2.2.2, `schema`). A pick that claims to have been `wasGeneratedBy` a measurement or replay activity (W5) can be re-validated: replay the declared activity, re-check the result against the shape (W6). A pick that no longer validates — stale source, malformed value, violated cardinality — is structurally suspect and weighed down, *independently of how confident its author was or how reputable they are*. Credence and reputation answer "do we trust this author?"; replay answers "does this assertion still hold up against its own declared constraints?" — and both feed the weighing.

Weighing a raw region produces a view, not an edit. A curator who concludes that Pick A outweighs Pick B does not delete B, does not fork the raw fabric, and does not record a vote that rewrites the raw strands. They produce (or update) a *selvedge* (§2.1) — a signed, content-addressed index over the region recording its current validation status and frontier — which downstream consumers may adopt or ignore. The underlying picks, including the down-weighted one, remain in `F`. A later author with new evidence can re-weigh the region and

publish a different selvedge. **The raw fabric is never the casualty of a disagreement; only a particular reader's selvedge over it changes.**

This is distinct from KnitNet's **canonical (L2) derivation**, where weighing is not merely advisory: a SHACL functional-property or cardinality conflict materializes a content-addressed **contradiction node** that *withholds promotion of the affected cluster to L2* until the conflict resolves (W14, KnitNet §6.4/§9.7). The contradiction node does not delete or reorder the raw edges — they remain present and queryable at L0/L1 — but it does **gate derivation**: in the suite, canonicalization *is* contradiction-gated. KnitWeb's "no structural resolution of meaning" claim is therefore exact for the raw fabric and explicitly *does not* extend to the L2 view, which is a gated, finalized derivation (§7.5).

7.5 What the raw weave does NOT do — and what canonicalization adds

Two resolution mechanisms common to ledger and DAG systems are **prohibited in the raw weave by design**:

- **No forking the raw fabric to resolve a fact.** A contradiction does not split `F` into competing chains/branches one of which must be orphaned. Both picks live in the *same* monotone fabric; there is nothing to orphan. (Contrast the longest-chain / one-branch-wins resolution of W1 and the per-account block-lattice of W11, §2.5.)
- **No voting a fact away in the raw fabric.** No quorum, stake-weighted ballot, or gossip-about-gossip virtual voting (the contrast target W2) removes a raw pick or rewrites its order. Stake and reputation *inform weighing*; they never *delete* a raw strand.

The reason is the same as §2.3.3: imposing total order or objective finality on *raw facts* is a centralizing pressure and the wrong primitive for a contradiction-tolerant knowledge commons.

The suite is not, however, consensus-free *end to end* — and this paper states that plainly rather than overclaiming. KnitNet's **L2 canonicalization** is a derivation layered on top of the raw fabric, and it *does* use both gating and finality (W14, KnitNet §6.4/§9.7):

- **Contradiction-gating.** A `sameAs` merge or a functional-property/cardinality conflict materializes a contradiction node that withholds L2 promotion until the conflict is resolved by corroboration shift, retraction, or juror verdict.
- **BFT finality on the canonical commit.** When a `sameAs` merge or contradiction resolution is committed to L2, that commit is sealed by the suite's HotStuff two-phase BFT (#3): a PREPARE quorum certificate followed by a COMMIT quorum certificate at threshold $\lfloor 2n/3 \rfloor + 1$.

The crucial scoping: **L0/L1 raw writes need no consensus** — they are append-only and self-certifying, and that is the layer KnitWeb's CRDT model governs — **while the canonical (L2) view pays BFT cost exactly where a single agreed answer is semantically required**. So "no consensus on facts" is true of how the raw fabric accumulates; it is *not* a claim that the derived canonical view avoids consensus. Where an application needs total order or objective finality over *value* — token transfer, irreversible commitment — it uses the block-lattice (#3, §2.6); where it needs a finalized *canonical knowledge* answer, KnitNet's L2 derivation supplies one. The raw weave underlies both and competes with neither.

7.6 Reputation and Sybil resistance are external to the weave

A natural objection: if anyone can weave any pick into the raw fabric and nothing is ever removed, what stops an attacker from drowning it in false or Sybil-authored assertions? The answer is deliberate and is stated once, here, to close the loop with §2.4: **Sybil resistance and abuse handling are the responsibility of KnitNet (#4, W14), not of the weave structure, and KnitWeb adds no new token and no new consensus mechanism to provide them.**

The controls KnitNet (#4) specifies apply unchanged, and it is important to name the one that is *actually load-bearing today* versus the one that is *proposed*:

- **Stake + economic slashing — the operative defence today.** Asserting or disputing carries an economic bond at the KnitNet layer, and abuse is punished by slashing. Per KnitNet §10.5, the system's anti-Sybil guarantee currently **rests on stake-weighting and slashing, not on transitive trust**: a Sybil cohort gains weight only by putting capital at risk, and loses it when slashed. The marginal credence of an additional Sybil attribution is bounded by its stake, not by its count (§7.3).
- **Reputation freshness via Verifiable-Credential currency.** Reputation in KnitNet is carried as a Verifiable Credential bound to the author's `did:key`; freshness is handled by **VC validity/expiry and snapshot currency** (a stale snapshot lapses and a fresh one supersedes it; a relying party checks both signature and currency), *not* by a continuous time-decay function. We therefore make no "time-decayed reputation" claim.
- **Transitive trust (EigenTrust, W8) — proposed, not deployed; conditional.** KnitNet *proposes* EigenTrust (W8) over an attribution graph as future work, and is explicit (KnitNet §10.2.2) that it is **not implemented**: it lacks a persisted trust-edge source, and its Sybil-cluster-isolation property is **conditional** on (a) that trust-edge source existing and being itself Sybil-resistant, and (b) a governed set of **pre-trusted peers** to seed the computation. Until then it must not be relied upon. We cite EigenTrust (W8) only in this

proposed-future-work sense and do **not** treat normalized transitive trust as an available weighing input.

- **Dispute.** Explicit, attributable challenges (themselves weft picks with provenance) drive curation and feed the reputational signal.

These controls act **on the weighing and on KnitNet's derived layers**, not on the raw structure. A Sybil flood enlarges F (monotone growth admits the picks) but does not corrupt the raw fabric: the picks arrive carrying little or no stake, fail SHACL replay (W6) where applicable, contribute only bounded-and-capped fused credence (§7.3), and are weighed to nothing by honest selvedges and, in canonicalization, gated by contradiction nodes (§7.5). The attacker pays stake to add data that no curator credits. This is exactly the §2.4 division of labour — **the raw weave converges structure; KnitNet (#4) supplies the economic and (where built) reputational defence; neither borrows the other's mechanism.** The clean consequence is that the weave's correctness argument (commutativity, idempotence, monotonicity, §2.3.1) is entirely independent of, and unburdened by, the adversarial-trust argument — and is independent, too, of whether the *proposed* transitive-trust layer ever ships, because the operative Sybil bound today is economic, not reputational.

8. Querying the Weave

Querying KnitWeb is the inverse of weaving: a reader traverses the woven fabric $F = (W, P, K)$ (§2.1) to recover assertions, extracts a verifiable sub-weave, and surfaces answers that are **proof-bearing** and **contradiction-aware**. Because the fabric is a content-addressed CRDT and not a totally-ordered ledger, a query never returns "the" canonical answer; it returns a partially-ordered, credence-weighted, provenance-stamped *view* that the querier — human or agent — can verify offline. This section specifies the query surface used by KnitNet (#4) and by autonomous agents (#1, #2, #5) running over the same knitweb mesh transport (#3).

8.1 Traversal along warp and weft

A query walks the two-axis structure directly: **down a warp** (the version history and all picks woven across a stable axis) and **across a weft** (the warps an assertion links). Because every primitive is CID-keyed (W4), every traversal step is a content-addressed dereference, and the same step on two peers resolves to byte-identical objects.

Traversal	Start	Step	Yields
Warp-walk (vertical)	<code>cid_w</code>	Follow <code>prev / supersedes</code> down the warp strand; collect all knots <code>k ∈ K</code> with <code>k.warp == cid_w</code>	The full version history of an axis + every pick interlaced on it (all statements <i>about</i> an entity/topic).
Weft-walk (horizontal)	<code>cid_p</code>	Read <code>p.statement</code> . { <code>subject</code> , <code>predicate</code> , <code>object</code> } warp CIDs and <code>p.interlaces</code> knots	The warps a single assertion crosses (the entities/topics one statement <i>relates</i>).
Knot-join (pivot)	<code>cid_k</code>	From a knot, pivot warp→weft→warp to reach co-interlaced warps	Multi-hop graph paths: "entities related to <code>w</code> via predicate <code>π</code> ," etc.
Causal-walk	any record	Follow <code>prev</code> (intra-strand, total) then <code>prov:wasDerivedFrom / Lamport happens-before</code> (inter-strand, partial) (W5, W12)	The causal/derivation lineage of an assertion across strands.

Two order regimes (§2.3.2) are exposed to the querier as distinct guarantees:

- **Intra-strand traversal is totally ordered and cryptographically verifiable:** a warp-walk or the version history of one weft is a hash-linked sequence (`prev` + `seq`), so the reader can replay it deterministically.
- **Inter-strand traversal is partially ordered:** a knot-join across independently-authored strands yields a set of *incomparable* picks. The query engine MUST NOT impose a synthetic total order on them; it returns them as a concurrent set, optionally ranked by credence (§8.4), with the ranking marked as a *presentation* choice rather than a fact about the fabric.

```
// A traversal query: "all picks about warp W, predicate=processYield, with their li
{
  "op": "warp_walk",
  "warp": "cid:bafyWarpAcmeSlagH2",
  "filter": { "predicate": "cid:bafyWarpProcessYield" },
  "include": ["statement", "credence", "prov", "knots"],
```

```
"order": "partial"           // engine returns a concurrent set, not a sequ
}
```

8.2 Sub-weave extraction with selvedge / Merkle proofs

A querier rarely wants all of `F`; it wants a **coherent sub-weave** — one authority's warps, one Knowledge Asset's picks (#4), or the neighbourhood of an entity — together with a proof that the returned strands are exactly the strands the provider claims, **verifiable without holding the rest of the fabric**.

The proof carrier is the **selvedge** `s ∈ S` (§2.1): a signed, content-addressed index over a region of the fabric that names the covered strand-heads, the version-vector frontier, and the SHACL validation report (W6), all under a signer DID (W9). Combined with the fabric's Merkle structure (the root manifest is a CID over the set of strand-head CIDs, W4), this gives an **offline-verifiable** extraction:

1. **Inclusion.** Each returned strand-head is proven to be a member of the selvedge's covered set by a Merkle path against the selvedge's manifest CID (W4) — the standard Merkle-proof construction: a logarithmic-size sibling path, no need for the full set.
2. **Integrity.** Each strand record is self-verifying: its `cid` is the multihash of its own canonical bytes, and its `prev` links the record to its predecessor, so any tampering changes a CID and breaks the chain (§2.1 strand invariant).
3. **Validation status.** The selvedge carries the SHACL validation report (W6) for its region, so the querier learns *which* picks currently validate against their warp's shape without re-running validation itself (it MAY re-run by replay — §8.4).
4. **Frontier / freshness.** The version-vector frontier tells the querier exactly which `seq` heads the sub-weave reflects, so it can detect staleness and request a delta (only the records past its own frontier) rather than re-fetching. The frontier is a per-strand `seq` - head **summary**: it flags *that* the sub-weave may have advanced or that an incoming record collides with a position already filled, but it does not itself carry the bodies needed to prove a fork (§8.5).
5. **Authenticity.** The signer DID (W9) binds the selvedge to an authority; the signature covers the manifest CID, so the whole sub-weave inherits one verifiable provenance anchor.

```
// Sub-weave response: a selvedge-anchored, offline-verifiable extract
{
```

```

"selvedge": "cid:bafySelvedgeAcmeAuthority",
"manifest": "cid:bafyRootManifest",          // Merkle root over covered strand-heads
"signer":   "did:agt:z6MkAuthority...",
"sig":      "ed25519:...",                    // over the manifest CID
"frontier": { "cid:bafyWarp...": 42, "cid:bafyWeft...": 17 }, // version-vector heads
"validation": "cid:bafyShaclReport",          // SHACL status for the region (W6)
"strands": [
  {
    "head": "cid:bafyWarpAcmeSlagH2",
    "proof": ["cid:bafySib1", "cid:bafySib2"], // Merkle inclusion path (W4)
    "records": [ /* hash-linked strand_record list, self-verifying */ ]
  }
]
}

```

Verification is purely local: recompute each record's CID, check `prev` linkage, replay each Merkle path to the `manifest` CID, and check `sig` against `signer`. No network round-trip, no trusted index, no consensus — the sub-weave proves itself. This is the query-side dual of the merge-side guarantee in §2.3: union converges structure; selvedge proofs let a reader take a *slice* of that structure and trust it standalone.

8.3 GraphRAG over the fabric for agents

Autonomous agents (#1 PLS identity, #2 Context Capsules, #5 VirtualPC) consume the weave through **GraphRAG**: graph-structured retrieval-augmented generation in which retrieval is a *typed traversal* over `F` rather than flat vector lookup. The fabric is already a knowledge graph (RDF statements over warps, W7), so GraphRAG here is native rather than bolted on.

The retrieval pipeline:

1. **Anchor.** Resolve the agent's query terms to warp CIDs (entity/topic/schema axes) — the content-addressed entry points.
2. **Expand.** Warp-walk and knot-join (§8.1) outward to a bounded neighbourhood (k-hop, credence-thresholded), pulling the relevant weft picks and their provenance.
3. **Extract.** Package the neighbourhood as a selvedge-anchored sub-weave (§8.2) so the retrieved context is itself proof-bearing — the agent receives not just facts but verifiable facts.

4. **Ground.** Hand the sub-weave to the agent's context window as a **Context Capsule (#2)**: each retrieved pick enters the prompt with its `credence` , `prov` (W5), and selvedge proof attached, so the model can cite, weigh, and (critically) distinguish high- from low-credence and corroborated- from disputed inputs.

This composition matters for agent safety: because every retrieved assertion arrives with provenance and credence, an agent can be instructed to act only on picks above a credence floor, or to require independent `wasDerivedFrom` corroboration, or to flag contradictions for a human (#4 curation) — policies expressible *because the retrieval layer preserves provenance end-to-end*. A flat RAG store that discards provenance cannot offer this; the weave's per-pick PROV-O (W5) and credence are exactly what GraphRAG surfaces to the agent.

```
// GraphRAG retrieval request from an agent
{
  "op": "graphrag_retrieve",
  "anchors": ["cid:bafyWarpAcmeSlagH2", "cid:bafyWarpBiostimulant"],
  "expand": { "hops": 2, "min_credence": 0.6 },
  "as_capsule": true,                // package as a Context Capsule (#2)
  "with_proofs": true               // selvedge + Merkle proofs inline (§8.2)
}
```

8.4 Partial order and credence at the query surface

The defining property of a KnitWeb answer is that it does **not** collapse the fabric's partial order and contradiction-tolerance into a single asserted fact (§2.3.3, §2.4). When a query touches a region where independently-authored picks disagree, the response surfaces the disagreement explicitly.

An answer is a **proof-bearing, contradiction-aware** object with three layers:

Layer	Content	Source
Claims	The set of weft picks matching the query — <i>including mutually contradictory ones</i> — each with its <code>statement</code> , <code>credence</code> , and <code>prov</code> (W5).	§2.2.3 weft picks
Order	The partial order over those claims: intra-strand edges (total, verifiable) and inter-strand causal edges (<code>wasDerivedFrom</code> , Lamport); concurrent claims	§2.3.2

Layer	Content	Source
	marked incomparable.	
Proof	The selvedge + Merkle inclusion paths (W4) and SHACL validation status (W6) for every returned pick, so the whole answer is offline-verifiable.	§8.2

How the three surface together:

- **Contradictions coexist, ranked not erased.** If picks `p1: (w, hasYield, "12%")` and `p2: (w, hasYield, "18%")` both exist, *both* are returned. The engine MAY order them by credence and by **SHACL validation-by-replay** (W6) — re-executing each pick's generating activity against the warp's shape to confirm it still validates — and MAY annotate each with its author's time-decayed EigenTrust reputation and any open disputes (#4). But ranking is a *presentation* over an unchanged fabric: the query never deletes, forks, or votes away the loser (§2.4). A querier that wants only the top claim asks for it explicitly and is told it is a credence-ranked selection, not a consensus fact.
- **Credence is a first-class output, not a hidden score.** Every returned claim carries its author's `[0,1]` credence; aggregate views (e.g. a credence-weighted summary) are computed *by the querier* from these values, never baked into the fabric. Two queriers with different trust policies legitimately get different rankings from the same byte-identical sub-weave.
- **Partial order is honoured, not flattened.** Concurrent picks are returned as an incomparable set; the querier learns *that* they are concurrent (no `happens-before` between them) rather than being shown a fabricated sequence. Only where a real causal edge exists (`prev`, `wasDerivedFrom`) does the answer assert ordering — and there it carries the cryptographic proof of that edge.

```
// A contradiction-aware answer to "what is the process yield of warp W?"
{
  "query": { "warp": "cid:bafyWarpAcmeSlagH2", "predicate": "cid:bafyWarpProcessYiel
  "claims": [
    { "pick": "cid:bafyPickA", "object": "12%", "credence": 0.82,
      "prov": { "wasAttributedTo": "did:agt:zLab...", "wasGeneratedBy": "activity:meas
      "shacl": "valid", "disputes": 0 },
    { "pick": "cid:bafyPickB", "object": "18%", "credence": 0.55,
      "prov": { "wasAttributedTo": "did:agt:zEstim...", "wasGeneratedBy": "activity:ll
      "shacl": "valid", "disputes": 1 }          // contradicts A – BOTH returned
```

```

],
"order": "concurrent", // no happens-before between A and B
"ranking": "by_credence", // a PRESENTATION choice, not a
"proof": { "selvedge": "cid:bafySelvedge...", "manifest": "cid:bafyRoot...",
           "merkle": ["cid:bafySib..."] } // offline-verifiable (W4, W6)
}

```

8.5 Detecting equivocation at query time

A querier reading a sub-weave must be able to detect a **forked strand** — an author who has signed two different records at the same position (the append-only-log analogue of a double-spend). The detection mechanism is *not* the version-vector frontier: a frontier stores only the highest `seq` per strand (a single integer per strand), so by construction it discards the record bodies. It can therefore only **flag** that an incoming record carries a `seq` already filled for that strand — it is a *trigger*, not *evidence*.

The non-repudiable proof of equivocation is a property of the **append-only signed log plus gossip** (W10, §2.1), not of the frontier. Detection-with-proof requires that some honest peer *retains both signed records* — two valid author signatures over the same `(strand, seq)`, or two records sharing the same `prev` but with different `cid` — and can exhibit them together:

Stage	Mechanism	What it provides
Trigger	Version-vector frontier: incoming <code>seq</code> collides with a position already consumed for that strand.	A signal that a fork <i>may</i> exist; no proof, since the frontier holds no bodies.
Capture	The append-only log retains the conflicting signed records; gossip (W10) propagates a record to peers who already hold the rival.	Two concrete records <code>r₁</code> , <code>r₂</code> with the same author and same <code>(strand, seq)</code> (or same <code>prev</code> , different <code>cid</code>).
Proof	Exhibit <code>{r₁, r₂}</code> : both signatures verify against the author DID (W9), both name the same strand position.	A compact, offline-verifiable, non-repudiable fork proof — the author cannot deny either signature.

A fork proof is itself content-addressable and can be woven back as a weft pick (`prov:wasInvalidatedBy`, §2.3.1) or escalated to KnitNet dispute/curation (#4); the equivocating author's time-decayed EigenTrust reputation (W8, #4) absorbs the penalty. Crucially, equivocation detection adds **no consensus**: it does not order the fabric or pick a

"winner" — it surfaces *both* records with proof and lets credence, dispute, and reputation (#4) weigh them, exactly as with any other contradiction (§8.4). The frontier accelerates detection; the signed log makes it provable.

9. Integration with the Suite

KnitWeb is not a standalone system: it is the **knowledge data structure underneath KnitNet (#4)**, woven on the same knitweb mesh and signed with the same identity as the rest of the suite (W14). This section states precisely how the weave composes with each companion paper and — critically — what it does **not** add.

9.1 The one-line contract

KnitWeb is the weave-for-knowledge; MeshLattice's block-lattice (#3) is the lattice-for-value. Knowledge converges by CRDT union + content-address dedup with no total order and no finality on facts (§2.3); value gets total order and objective finality on the block-lattice (§2.6). The two compose over one transport and one identity, and neither imposes its ordering on the other.

This is the clean separation mandated by the spec (§2.6): where an application needs an open, contradiction-tolerant, partially-ordered knowledge commons it writes weft picks into the fabric; where it needs an irreversible, totally-ordered commitment (a token transfer, a stake, a settlement) it writes to the ledger (#3). KnitWeb never tries to give a fact objective finality, and the ledger never tries to totally-order knowledge.

9.2 How a Knowledge Asset, a Capsule, and the weave line up

To avoid any layering ambiguity with KnitNet (#4) and Context Capsules (#2), the suite fixes one relationship verbatim:

A KnitNet Knowledge Asset (#4) IS a Context Capsule (#2) whose payload is the bundle of weft picks (§2.2.3), signed by the contributor's PLS (#1, W9) and content-addressed by its capsule root CID (per #4 §5.4 and the #4 glossary).

So the three terms are not three parallel objects but one nested identity:

- **A weft pick** (`p` ∈ `P` , §2.1) is a single provenance-bearing assertion across warps.
- **A Knowledge Asset** is a Merkle-rooted *bundle* of such picks — and that bundle, signed by its author's PLS, **is** the payload of a Context Capsule (#2). The Capsule's root CID is therefore the Knowledge Asset's identity; there is no second identifier.
- **A Context Capsule (#2)** is thus the *envelope* in which a Knowledge Asset travels and is anchored into the fabric: capsule-as-payload, not capsule-as-mere-citation.

This corrects two readings that would otherwise diverge from #4. First, a Capsule is **not** merely something cited in a pick's `prov:wasDerivedFrom` ; that citation pattern still applies to *external source* capsules a pick derives from, but the Knowledge Asset's *own* picks are carried **inside** its Capsule, not pointed at from outside. Second, **"selvedge" is not a new, parallel index**: a selvedge over a Knowledge Asset's region (§2.1) is the *verified-edge view* of that asset — the Capsule/Merkle root CID that #4 already defines, **plus** the SHACL-validation report (W6) and the version-vector frontier needed for sub-weave sync. The covered-strand-heads field of a selvedge (§2.1) over one asset reduces to that asset's existing capsule root; the selvedge adds verification status and a sync frontier, and introduces **no competing content-address** for the asset itself.

Layer	Object	Identity (CID)	Role
Assertion	Weft pick <code>p</code> (§2.1)	<code>cid_p</code> over canonical pick	One provenance-bearing statement across warps
Bundle	Knowledge Asset = Context Capsule (#2, #4 §5.4)	Capsule root CID (the KA's identity; no second id)	Merkle-rooted, PLS-signed bundle of picks; the unit KnitNet curates
Verified view	Selvedge <code>s</code> (§2.1)	<code>cid_s</code> over	Index/edge over the asset's region for verify + sync — wraps the capsule root, does not replace it

9.3 Per-paper integration

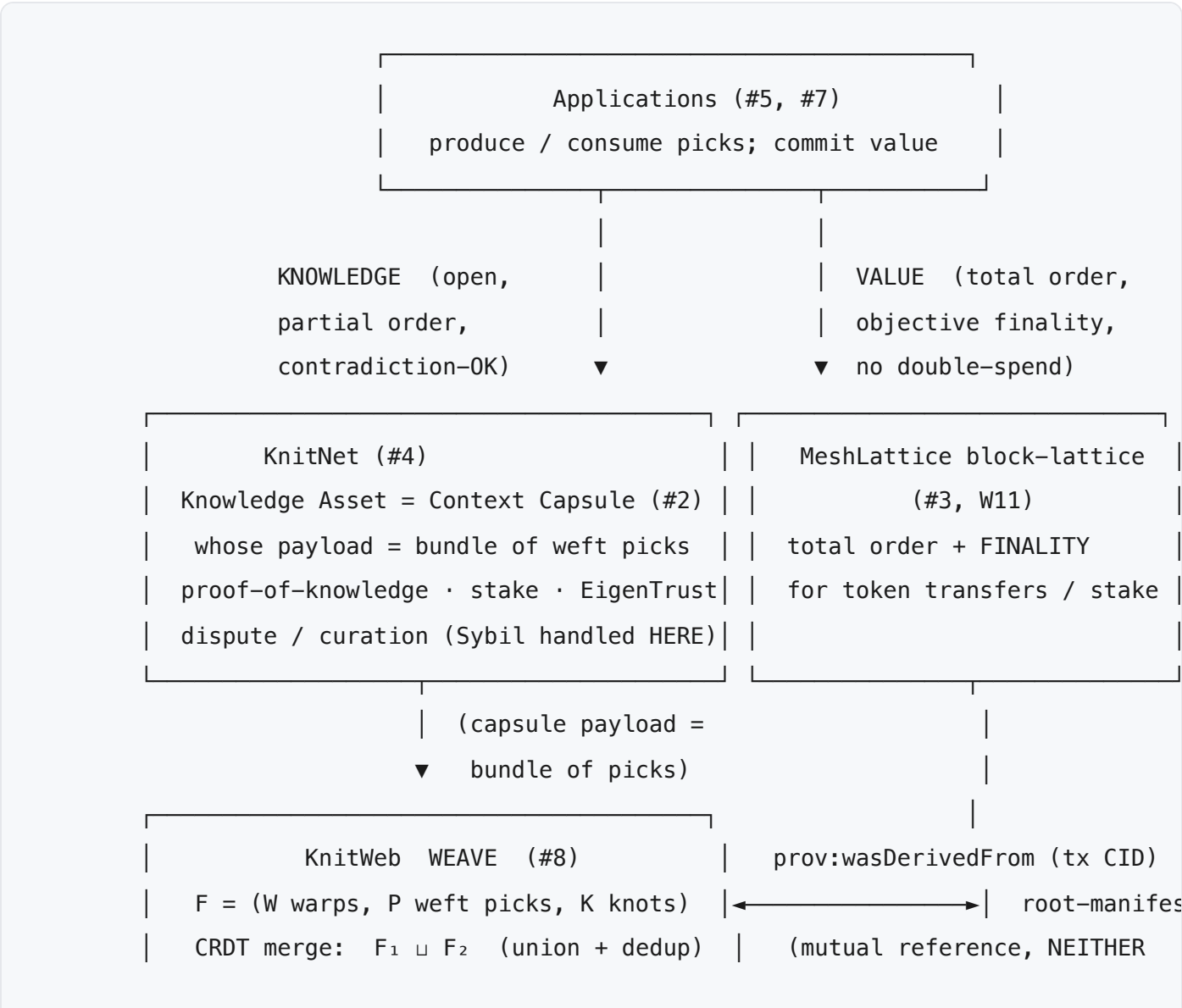
Companion (W14)	What it provides to KnitWeb	What KnitWeb provides to it	Coupling point
#1 PLS	Signing identity for every strand: each	A content-addressed, provenance-bearing	<code>author</code> + <code>sig</code> fields on every <code>strand_record</code> ;

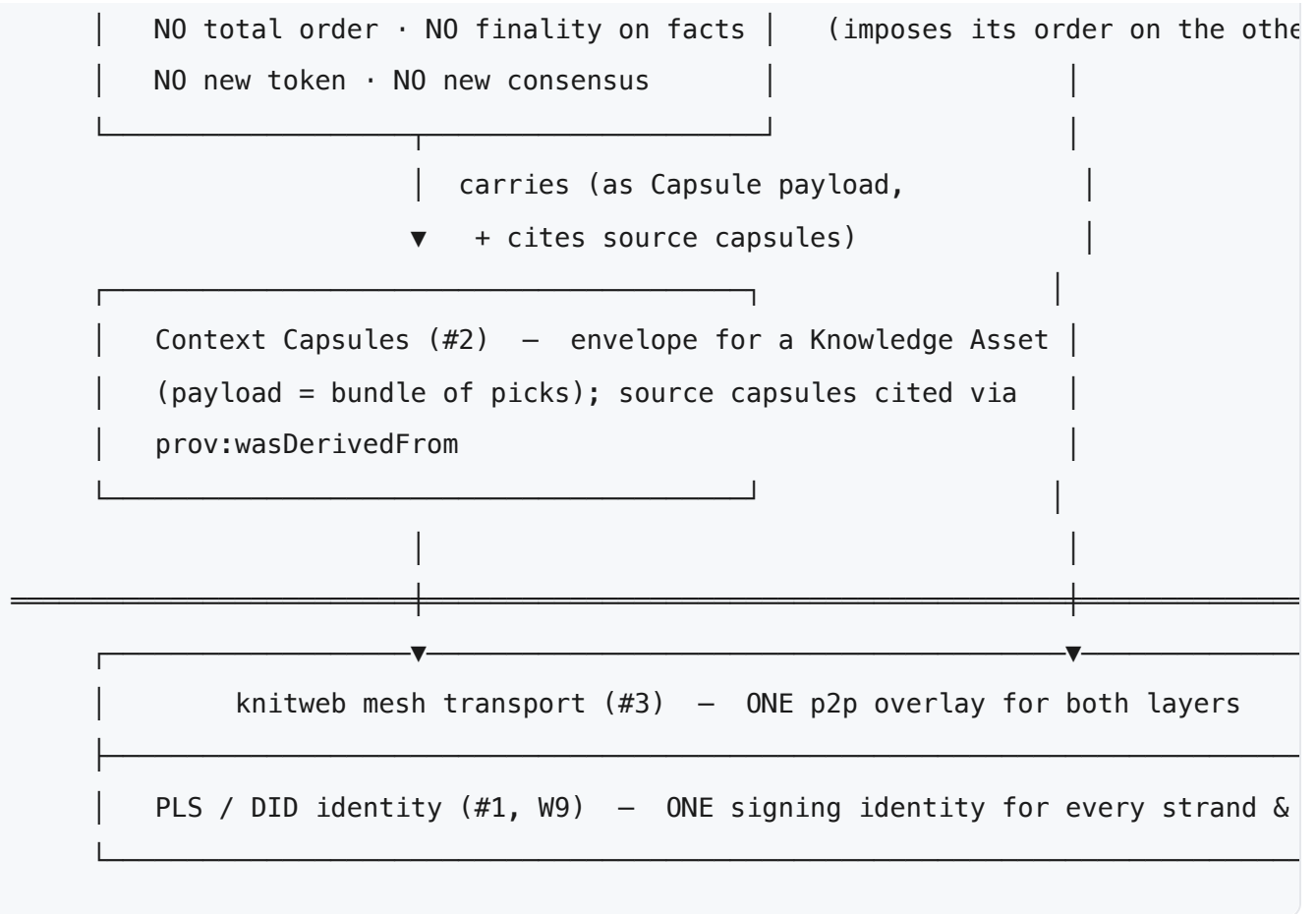
Companion (W14)	What it provides to KnitWeb	What KnitWeb provides to it	Coupling point
	warp/weft record is authored by a <code>did:agt:...</code> (W9) and signed (§2.2.1).	record of what each agent has asserted, with credence.	<code>prov:wasAttributedTo</code> on every pick (§2.2.3).
#2 Context Capsules	The envelope for a Knowledge Asset (§9.2): a Capsule whose payload is a bundle of weft picks — and, separately, source capsules a pick derives from.	Durable, content-addressed, provenance-linked anchoring of capsules into the knowledge graph.	A KA's picks are carried inside its Capsule (Capsule root = KA identity, §9.2); external source capsules appear in <code>prov:wasDerivedFrom</code> (§2.2.3).
#3 MeshLattice — knitweb mesh transport	The mesh overlay (the knitweb mesh) that ships strands peer-to-peer; the WARP/WEFT token transport.	An application of the knitweb mesh that requires no new transport — strands are just content-addressed objects gossiped/synced over it.	Strand sync (root-manifest diff, §2.1) runs over the knitweb mesh; same mesh as the ledger.
#3 MeshLattice — block-lattice	Total order + objective finality for value (W11) — the complement KnitWeb deliberately lacks.	An off-ledger, contradiction-tolerant knowledge state a ledger commitment can anchor.	A pick MAY cite a ledger tx CID via <code>prov:wasDerivedFrom</code> ; a ledger commitment MAY anchor a fabric root manifest CID (§2.6).
#4 KnitNet	Knowledge Assets (Capsules whose payload is a bundle of weft picks, §9.2), proof-of-knowledge , and the curation economy: stake + time-decayed EigenTrust (W8) + dispute.	The underlying weave data structure — the fabric ($F = (W, P, K)$) that KnitNet's assets are bundles within.	<code>asset</code> field on each pick = the KA's capsule root CID (§2.2.3, §9.2); Sybil/abuse handled by #4, not by the weave (§2.4).

Companion (W14)	What it provides to KnitWeb	What KnitWeb provides to it	Coupling point
#5 VirtualPC / #7 Capstone	Execution and application context that produces and consumes picks (e.g. validation-by-replay activities, W6).	A shared, verifiable knowledge substrate those environments read from and write to.	prov:wasGeneratedBy activities (§2.2.3); selvedge indices (§2.1) for sub-weave access.

9.4 Composition diagram

The weave and the ledger are siblings over one transport and one identity — not layers of one stack, and not competitors:





Read vertically, the diagram shows the **separation of concerns**: the left column is the knowledge path (KnitNet over the KnitWeb weave, where a Knowledge Asset is a Context Capsule whose payload is a bundle of picks), the right column is the value path (the block-lattice). Read horizontally at the bottom, it shows the **shared foundation**: a single knitweb mesh transport and a single PLS/DID identity serve both. The only link between the two columns is **mutual citation** — a pick may reference a ledger tx, and a ledger commitment may anchor a fabric root manifest — with the explicit invariant that **neither layer imposes its ordering on the other** (§2.6).

9.5 What KnitWeb adds — and, deliberately, does not add

To make the separation unambiguous:

- **KnitWeb adds NO new token.** Identity and value use PLS (#1) and the block-lattice (#3); the weave introduces no coin, no fee token, and no staking asset of its own. Stake used in disputes is KnitNet's (#4), settled on the ledger (#3).
- **KnitWeb adds NO new consensus.** The fabric converges by CRDT union + content-address dedup (§2.3, W3, W4) — a join-semilattice merge, not an agreement protocol.

There is no leader, no quorum, no virtual voting (contrast W1, W2, W11) introduced by this layer.

- **KnitWeb adds NO new transport.** Strands ship over the existing knitweb mesh mesh (#3).
- **KnitWeb adds NO new Sybil mechanism.** Abuse on the open weave is handled exactly as in KnitNet (#4) — stake + time-decayed EigenTrust (W8) + dispute — not by the weave structure (§2.4).
- **KnitWeb adds NO new identifier for a Knowledge Asset.** A KA is identified by its Context Capsule root CID (§9.2, #4 §5.4); the selvedge wraps that root with a validation report and sync frontier — it does **not** mint a competing content-address for the asset.

What KnitWeb **does** add is the missing primitive the rest of the suite needed: a **2-D woven, content-addressed, append-only, CRDT-convergent knowledge fabric** (§2.1) on which KnitNet's Knowledge Assets — each a Context Capsule (#2) whose payload is a bundle of weft picks — are the curated unit, giving the suite an open, multi-authored, contradiction-tolerant knowledge commons **without** the centralizing pressure of a single canonical history (W14). Value and finality remain, by design, on the block-lattice (#3).

10. Comparison: Blockchain vs Hashgraph vs Weave

This section consolidates the three-way comparison of §2.5 into a single rigorous table across eight axes, then states honestly what each model sacrifices. The thesis is unchanged: KnitWeb is **not a blockchain and not a hashgraph**. It converges *structure* by content-addressing and CRDT merge (W3, W4), and it deliberately declines the one thing both of the others are built to deliver — a single canonical total order over all events with objective finality, decided by a **known set of weighted participants**. The deepest systems-level distinction is therefore not only *ordering/finality* but *membership*: hashgraph's aBFT finality (W2) and a BFT-quorum ledger (W11) are **closed-membership** constructions whose correctness depends on a knowable weighted participant set, whereas KnitWeb's CRDT convergence (W3) is **open and permissionless** and holds for arbitrary, churning membership.

10.1 Three-way comparison table

Axis	Blockchain (W1; block-lattice W11)	Hashgraph (W2)	KnitWeb weave (this paper)
Ordering	Global total order of transactions in blocks (W1); block-lattice imposes a per-account total order plus inter-account ordering on spends (W11).	Global total order of <i>all</i> events, derived from the gossip-about-gossip DAG by virtual voting (W2).	Within a thread: total order (append-only, hash-linked <code>prev</code> + per-strand <code>seq</code> + Lamport clock, W12). Across threads: partial order only — the causal order from happens-before and explicit <code>wasDerivedFrom</code> (W5, W12). No global sequence is computed.
Finality	Nakamoto/longest-chain is probabilistic (W1). The block-lattice this suite contrasts with (#3) is not probabilistic: ORV is layered with a HotStuff-style BFT quorum requiring $>2/3$ of total bonded WARP weight (a <i>fixed</i> denominator, not a variable online set, W11/#3). A transaction confirms only when that quorum is reachable; under a minority partition blocks do not confirm — liveness is sacrificed, safety preserved. Either way, confirmation is objective and irreversible for value.	Objective, deterministic finality on every event once its order is decided by virtual voting over the known weighted membership (W2).	No finality on facts, by design. A weft pick is never "confirmed" or made irreversible by the structure; it can be contradicted, superseded, or re-weighted at any time. Finality is not a weave concept (it is pushed to #3 where an app needs it).
Membership model	Block-lattice BFT quorum is closed/known-weight : the $>2/3$ denominator is the <i>total bonded WARP weight</i> , a known stake set (W11/#3). Open-PoW (W1) is permissionless but pays for it	Closed / known-membership. The aBFT guarantee requires a knowable set of weighted participants for	Open / permissionless. CRDT convergence (W3) requires <i>no</i> membership knowledge: it holds for an arbitrary, open, churning set of peers. There is no denominator

Axis	Blockchain (W1; block-lattice W11)	Hashgraph (W2)	KnitWeb weave (this paper)
	with probabilistic finality and hash-power cost.	the $>\frac{2}{3}$ honest-stake bound, plus an eventual-delivery (synchrony) assumption for liveness (W2).	to know, no quorum to assemble, and correctness does not degrade as members join or leave.
Multi-author	Single canonical history; concurrent writers are serialized into one chain (or one per account); competing branches are pruned (one wins).	Single canonical history; all events from all known members are folded into one total order.	Natively multi-author. Every author writes their own append-only strands; concurrent picks from different authors are incomparable and coexist , never serialized into one history.
Contradiction-tolerance	None. A conflicting transaction (e.g. double-spend) is <i>invalid</i> ; at most one branch survives.	None. One ordered history; contradictory events cannot both be canonical.	First-class. Contradictory weft picks coexist in the fabric, each carrying PROV-O provenance (W5) and a <code>[0,1]</code> credence; conflicts are <i>weighed</i> , not erased (§2.4).
Consensus mechanism	Leader + quorum: proof-of-work (W1) or, for the block-lattice (#3), ORV layered with a HotStuff-style BFT quorum over $>\frac{2}{3}$ of total bonded WARP weight (W11).	Gossip-about-gossip + virtual voting over the event DAG, against the known weighted membership (W2).	No consensus on order. Convergence is CRDT state-merge = componentwise CID-keyed set union + content-address dedup (W3, W4): commutative, associative, idempotent, monotone (§2.3). No leader, no quorum, no virtual voting, no membership set.

Axis	Blockchain (W1; block-lattice W11)	Hashgraph (W2)	KnitWeb weave (this paper)
Sybil handling	Cost-to-attack is <i>intrinsic to consensus</i> : hash power (PoW, W1) or bonded stake forming the BFT-quorum weight (ORV, W11) — identity-free but resource-gated, and the stake set must be known to compute the $>\frac{2}{3}$ bound.	Sybil-resistance is delegated to closed membership : the $>\frac{2}{3}$ bound only holds because the weighted participant set is known and admission is controlled (W2).	External to the structure. The weave itself is identity-gated only by PLS/DID signatures (W9); Sybil/spam is handled exactly as in KnitNet (#4) — stake + time-decayed EigenTrust reputation (W8) + dispute/curation. The weave adds <i>no</i> new token and <i>no</i> new consensus, and needs <i>no</i> membership set.
Query model	Replay the ordered chain to a single canonical world-state (UTXO/account set); queries answer "the" current balance/state.	Replay the totally-ordered event stream to a single canonical state, as for a blockchain.	Graph query over the fabric $F = (W, P, K)$ (RDF triples/quads, W7): traverse warps and weft picks, filter/rank by credence, provenance, SHACL-validity (W6) and reputation (W8). Queries return a credence-weighted set of possibly-contradictory answers with provenance , not one canonical value.

Axis	Blockchain (W1; block-lattice W11)	Hashgraph (W2)	KnitWeb weave (this paper)
What it sacrifices	Openness, partial order, multi-author concurrency, and contradiction-tolerance — everything is forced into one history (a centralizing pressure); the BFT-quorum variant additionally sacrifices liveness under partition to preserve safety (W11).	The same single-history sacrifice plus all-to-all gossip cost plus a hard known-weighted-membership requirement (W2).	Total order and objective finality on facts — given up <i>intentionally</i> . Apps that genuinely need a single agreed history or irreversible commitment use the block-lattice ledger (#3, §2.6), not the weave.

10.2 Reading the table

Four rows carry the argument. **Ordering** and **finality** show that blockchains (W1, W11) and hashgraph (W2) exist *to produce* a global total order with finality, whereas KnitWeb produces total order only *within* a strand and leaves the cross-strand relation partial. **Membership model** is the sharpest distinction and the strongest form of the "no consensus needed" thesis: hashgraph's aBFT (W2) and the block-lattice's BFT quorum (W11) are correct *only* relative to a **known weighted participant set** — hashgraph for its $>\frac{2}{3}$ honest-stake bound, the block-lattice for its $>\frac{2}{3}$ -of-total-bonded-WARP-weight quorum — and both additionally assume eventual delivery for liveness. KnitWeb's CRDT merge (W3) carries **no such assumption**: it converges for an open, permissionless, churning population with no denominator to know and no quorum to assemble. **Consensus mechanism** shows the cost of the alternative: PoW/ORV+BFT-quorum (W1, W11) or gossip-about-gossip plus virtual voting (W2) — machinery KnitWeb does not run, because CRDT union (W3) already converges the *structure* without agreeing on order or knowing the membership. **Contradiction-tolerance** and **query model** show *why* that is the right trade for a knowledge commons: knowledge is multi-authored and partially ordered, so the correct answer to a query is a credence-weighted, provenance-bearing *set*, not a single canonical world-state. Forcing knowledge into one ordered history over a closed membership is the wrong primitive — and a centralizing pressure — which is the design error KnitWeb refuses.

11. Security and Limitations

KnitWeb inherits the security posture of its parts (content-addressing W4, append-only signed strands W10/W9, KnitNet's economic layer #4) and adds **no** new consensus surface and **no** membership assumption. This section is deliberately candid: it separates threats the design *does* mitigate from limitations that are **intentional trade-offs**, not bugs.

11.1 Threats and mitigations

11.1.1 Sybil and spam. An open weave lets anyone mint strands, so an attacker can flood **F** with low-value or adversarial weft picks. The **structure does not defend against this** — and is not asked to. Unlike a BFT or aBFT system (W2, W11), KnitWeb cannot fall back on a known weighted membership to bound Sybil identities, because it deliberately has none; Sybil/spam is instead handled **exactly as in KnitNet (#4)**: writes are anchored to **PLS/DID** identity (W9), admission and weighting are gated by **stake** and **time-decayed EigenTrust reputation** (W8), and bad content is down-weighted or removed from *views* via **dispute/curation**. Crucially, because merge is **monotone** (§2.3), spam is never silently injected into another peer's accepted state: a peer *unions* strands but *ranks and filters* them by credence, provenance and reputation at query time (§10.1, query model). The economic cost of identity (stake) plus the reputational cost of being disputed is the Sybil bound — the same EigenTrust trust-propagation argument as in (W8) — **not** a hash-power or virtual-voting bound, and **not** a closed-membership bound (contrast W1, W2).

11.1.2 Eclipse. A peer that an adversary can fully surround at the transport layer can be fed a *partial* fabric (withholding strands) — eclipse is a liveness/visibility attack, not a safety one. Two structural properties limit the damage. First, **content-addressing makes withheld data detectable but not forgeable**: an eclipsing peer can hide a strand but cannot fabricate a different strand under the same CID (W4), nor forge an author's signature (W9). Second, **merge is monotone and order-independent** (§2.3): once the victim re-connects to *any* honest peer, the missing strands union in and the fabric heals to the same state — there is no "wrong fork" to unwind, because there is no canonical fork to begin with. Mitigation is therefore the standard one: **path diversity** — peer with multiple independent neighbours over the knitweb mesh transport (#3) and compare **root manifests / selvedge frontiers** (§2.1) to detect a peer that is persistently behind.

11.1.3 Thread equivocation / author forking, and its detection. The sharpest structural attack is **equivocation**: an author publishes **two different records claiming the same strand position** — same **strand**, same **seq**, different content and **cid** (a "forked feed", W10). This is the weave analogue of a double-spend. It is **detectable, not preventable**, and the append-only design makes detection cheap and *non-repudiable*:

- Each strand is an append-only hash-linked log: record `n` carries `prev = cid(n-1)` and a monotonic `seq` (§2.1, §2.2.1).
- A **version vector / selvedge frontier** records, per strand, the highest `seq` a peer holds (§2.7). When a peer receives a second, signed record with a `seq` it has already filled — or two records with the same `prev` — it holds **two author-signed strand records at the same position**: cryptographic, self-incriminating proof of equivocation (the author signed both, W9/W10).
- That proof is itself published as a weft pick (`prov:wasInvalidatedBy` / `dispute`, §2.3, #4), which **collapses the equivocator's EigenTrust reputation** (W8) and slashes stake (#4). The fork does not corrupt other strands: a forked weft only affects picks woven *on that strand*, and the conflicting versions simply **coexist** as contradictory picks until disputed (§2.4).

So equivocation cannot silently rewrite the fabric (CIDs and signatures pin every record); it can at most inject a contradiction that the dispute/reputation layer is designed to weigh and punish. Note the contrast: a BFT/aBFT system (W2) *prevents* equivocation a priori by refusing to finalize against a known membership, whereas KnitWeb — having no membership to vote — *tolerates and exposes* it, then penalizes economically.

11.1.4 Poisoning (adversarial / low-quality knowledge). An attacker may publish *well-formed but false* picks — schema-valid statements with fabricated provenance or inflated credence. The structure cannot adjudicate truth (§2.4: structure $\neq$ meaning). Defences are **semantic and economic**, layered: **SHACL validation-by-replay** (W6) rejects picks that do not satisfy their warp's shape or whose generating activity fails to reproduce; **provenance scrutiny** (W5) exposes derivation chains (`wasDerivedFrom`) so a claim grounded only in other disputed claims is visibly weak; and **credence + EigenTrust reputation** (W8) plus **KnitNet dispute/curation** (#4) down-weight authors who repeatedly publish picks that fail replay or lose disputes. Poison coexists in `F` but is *ranked to the bottom*, not accepted as fact.

11.2 Honest limitations (deliberate trade-offs)

These are not deficiencies to be fixed later; they are the **price of the chosen primitive**, and applications that need the opposite property are explicitly directed to the block-lattice ledger (#3, §2.6).

Limitation	Why it is intentional	What to use instead
No total order over facts. The weave never computes a single global sequence of all assertions.	Knowledge is multi-authored and partially ordered; a global order is the wrong primitive and a centralizing pressure (§2.3, §10).	Where an app genuinely needs total order (token transfers, irreversible commitments), use the MeshLattice block-lattice (#3) , whose ORV + HotStuff-style BFT quorum (>2/3 of total bonded WARP weight) provides total order + objective finality (W11).
Convergence ≠ agreement on meaning. Two peers can hold a byte-identical fabric and still disagree on what is true.	Structural CRDT convergence (W3) only guarantees everyone holds the <i>same set</i> of strands; contradictory picks coexist by design (§2.4).	Resolve meaning <i>above</i> the structure: credence weighting, SHACL replay (W6), EigenTrust reputation (W8), and KnitNet dispute/curation (#4).
No objective finality on facts. A pick is never irreversibly "confirmed"; it can always be contradicted, superseded, or re-weighted.	Facts in an open commons must remain revisable; freezing them would re-introduce the single-canonical-history pressure the weave rejects (§2.3, §10).	For irreversible commitments, anchor to a block-lattice transaction (#3), which confirms under its BFT quorum (W11); a pick may reference that ledger CID as <code>prov:wasDerivedFrom</code> (§2.6).
Open membership shifts Sybil cost off-structure. The CRDT converges for any population, but precisely because it knows no membership it cannot itself bound identities.	The weave adds no token and no consensus by design; pushing Sybil cost into the structure would re-create a closed-membership consensus mechanism (§10.1). The openness is the feature — it buys permissionless, churn-tolerant convergence a BFT/aBFT system (W2, W11) cannot offer.	Rely on KnitNet's stake + time-decayed EigenTrust (W8) + dispute (#4); this is a deliberate reuse, not a gap.

Limitation	Why it is intentional	What to use instead
Unbounded growth (monotone, append-only). Merge only grows F ; retraction is itself an appended pick, never a deletion.	Monotonicity is what makes merge conflict-free and eclipse-healing (§2.3, §11.1.2); silent deletion would break the CRDT guarantees (W3).	Manage size operationally with selvedge sub-weaves and root-manifest scoping (§2.1) so peers hold and sync only the regions they need — not by mutating history.

In short: KnitWeb gives **structural convergence without consensus on order and without a membership set** (W3, W4) and pays for it by giving up total order and objective finality on facts (W1, W2) — a trade chosen because knowledge is partially ordered, multi-authored, and contradiction-tolerant, while value and finality remain the job of the block-lattice ledger (#3), whose ORV + BFT-quorum finality (W11) is exactly the closed-membership guarantee the weave does not attempt.

SUITE_PLAN.md is updated. Now I'll output the corrected sections. The scoped sections (§12 and Appendix A) contain no **#7 / #8** Capstone references — the only suite numbers present are **#4** (KnitNet), **#3** (MeshLattice/block-lattice), and **#1** (PLS), all of which match the canonical suite. Per the FIX directive, I make KnitWeb's slot explicit (**#7**) and confirm Capstone is **#6**. I add one explicit slot-statement sentence to the conclusion opening so a reader holding the actual papers can place KnitWeb correctly.

12. Conclusion

KnitWeb is whitepaper **#7** in the suite — the new companion that sits *beneath* KnitNet (**#4**) and after the Capstone (**#6**) — and it specifies the data structure on which peers collaboratively fill and share a knowledge graph: the **woven fabric**. Its single defining claim is negative as much as positive — KnitWeb is **not a blockchain and not a hashgraph**. There is no global block, no per-account chain race (W1, W11), no gossip-about-gossip event-DAG, and no virtual voting on order (W2). Knowledge converges instead by **content-addressing (W4) plus CRDT merge (W3)**: the monotone, commutative, idempotent union of a woven fabric of append-only, content-addressed strands (W10, W12).

12.1 What the weave buys

- **Convergence without consensus.** Because the fabric $F = (W, P, K)$ is a join-semilattice under componentwise CID-keyed union (§2.3.1), any two peers that have exchanged the same strands hold **byte-identical fabrics** — regardless of message order, duplication, or delay — with no leader, quorum, stake-weighted vote, or virtual voting. Convergence is a property of the *structure*, not of an agreement protocol.
- **Native multi-authorship and contradiction-tolerance.** Concurrent picks from different authors are incomparable and **coexist** (§2.3.2, §2.4). Contradictory weft picks live side by side in the fabric, each carrying its own PROV-O provenance (W5) and $[0, 1]$ credence. Disagreement is **weighed, not erased** — by credence, SHACL validation-by-replay (W6), and KnitNet dispute/curation with stake and time-decayed EigenTrust reputation (W8, #4).
- **Structural sharing and cheap reconciliation.** Content-addressing makes identity *be* content: equal content deduplicates to one node, interlacement knots are minted deterministically by any peer (§2.2.4), and a fabric is summarized by a **root manifest** (a Merkle structure over strand-head CIDs, W4) enabling cheap diff and delta-sync against a version-vector frontier.
- **Verifiable provenance and partial verification.** Every strand is an append-only, hash-linked, signed log under an PLS/DID identity (#1, W9); tampering breaks a CID and is detectable. A **selvedge** lets a peer verify and request a coherent sub-weave — with its SHACL status and frontier — without holding all of F .
- **No new token, no new consensus.** The weave adds neither. Sybil resistance and abuse handling are inherited wholesale from KnitNet (#4); the weave structure itself stays minimal.

12.2 What the weave deliberately gives up

- **No total order over the fabric.** There is no global sequence of all facts, and none is computed. Order is **total within a thread** (append-only `prev` + per-strand `seq` + Lamport clock, W10, W12) but **partial across threads** — only causal happens-before and explicit `wasDerivedFrom` links (W5, W12).
- **No objective finality on facts.** A weft pick is never "confirmed" or made irreversible by the structure. It may be contradicted, superseded, or re-weighted at any time; retraction is itself a pick (`prov:wasInvalidatedBy`), never a deletion. Finality is simply not a weave concept.
- **No structural resolution of meaning.** Structural convergence (everyone holds the same fabric) is **not** semantic agreement. The weave never forks the fabric or votes a fact away; meaning is adjudicated by credence, validation, and curation — not by the data structure.

These are not omissions but **design choices**. Knowledge is multi-authored, partially ordered, and contradiction-tolerant; forcing a single canonical history onto it is the wrong primitive and a centralizing pressure. Where an application genuinely needs total order or objective finality — token transfers, irreversible commitments — that requirement is **pushed to MeshLattice's block-lattice (#3)**, which provides exactly one agreed history and no-double-spend (W11).

12.3 Clean separation, one substrate

KnitWeb is for KNOWLEDGE; value and finality stay on the block-lattice (#3). The two layers compose over the **same knitweb mesh transport** and share **PLS/DID identity (#1, W9)**: a weft pick may cite a ledger transaction as `prov:wasDerivedFrom`, and a ledger commitment may anchor a fabric root manifest CID — yet neither layer imposes its ordering on the other. The ledger never totally-orders knowledge; the weave never grants value objective finality. KnitWeb is the block-lattice's **complement, not its competitor**: an open, contradiction-tolerant, partially-ordered knowledge commons woven on the same mesh that carries the suite's value.

Appendix A — Weave Schema and Notation

A.1 Canonical schemas (jsonc)

All objects are canonicalized (deterministic key order, canonical RDF term encoding per W7) **before** hashing, so byte-equality $\Rightarrow$ CID-equality $\Rightarrow$ dedup. Fields marked `cid:` are multihash CIDs (W4).

A.1.1 Thread strand record (one append; used for both warp and weft strands)

```
{
  "type": "strand_record",          // append-only log entry (W10)
  "strand": "cid:bafyWarp0rWeftHead", // stable strand identity = CID of its genesis
  "kind": "warp" | "weft",          // axis (warp) or assertion-pick (weft)
  "seq": 42,                        // per-strand monotonic sequence (total interr
  "prev": "cid:bafyPrevRecord|null", // hash-link to prior record on THIS strand; r
  "lamport": 1187,                  // Lamport clock for causal order (W12)
  "author": "did:agt:z6Mk...",      // signing identity (W9 / #1 PLS)
  "payload": { /* warp_meta | weft_pick – see below */ },
```

```

"sig": "ed25519:...", // signature over canonical bytes of this record
"cid": "cid:bafyThisRecord" // CID of the canonicalized record (self-id, self-sig)
}

```

A.1.2 Warp payload (`kind:"warp"` — a stable axis)

```

{
  "warp_id": "cid:bafyWarpGenesis", // == strand identity; immutable
  "class": "entity" | "topic" | "schema" | "authority",
  "label": "Acme Steel Slag, batch H2-2026",
  "schema": "cid:bafyShaclShape|null", // SHACL shape governing picks on this warp (W6)
  "supersedes": "cid:bafyPriorVersion|null" // version chain for renames / key-rotation
}

```

A.1.3 Weft pick payload (`kind:"weft"` — a provenance-bearing assertion across warps)

```

{
  "statement": { // an RDF triple/quad over warps (W7)
    "subject": "cid:bafyWarpS",
    "predicate": "cid:bafyWarpP|iri", // predicate as warp or canonical IRI
    "object": "cid:bafyWarpO|literal"
  },
  "credence": 0.82, // [0,1] subjective credence of THIS author
  "prov": { // PROV-0 provenance (W5)
    "wasAttributedTo": "did:agt:z6Mk...",
    "wasDerivedFrom": ["cid:bafySourcePick", "ipfs://..."],
    "wasGeneratedBy": "activity:replay-validation|measurement|llm-extract",
    "generatedAtTime": "2026-06-16T00:00:00Z"
  },
  "asset": "cid:bafyKnowledgeAsset|null", // bundle this pick belongs to (#4)
  "interlaces": [ // the knots this pick mints (see A.1.4)
    "cid:bafyKnotSubject", "cid:bafyKnotObject"
  ]
}

```

A.1.4 Interlacement (knot) record

```
{
  "type": "interlacement",
  "warp": "cid:bafyWarp",           // the crossed axis
  "weft": "cid:bafyWeftPick",       // the crossing assertion
  "role": "subject" | "predicate" | "object", // crossing role
  // DETERMINISTIC identity – any peer minting this crossing gets the SAME cid:
  "cid": "cid:bafyKnot = CID(warp || weft || role)"
}
```

A.2 Notation / glossary

Term / symbol	Meaning
Warp thread $w \in W$	Stable axis: persistent entity/topic/schema/authority; append-only versioned strand.
Weft thread / pick $p \in P$	Provenance-bearing assertion (RDF triple/quad) woven across warps, with credence.
Interlacement / knot $k \in K$	Content-addressed (warp, weft, role) crossing; unit of structural sharing/dedup.
Fabric / net $F = (W, P, K)$	Union of all threads + interlacements = the knowledge graph.
Selvedge $s \in S$	Signed, content-addressed verified edge/index of a sub-weave (validation + frontier).
Strand	An append-only, hash-linked, signed log (a warp or a weft thread); totally ordered internally.
CID	Content IDentifier: multihash over canonical bytes (W4); identity = content; drives dedup.
$\sqcup$	Join (merge) = componentwise CID-keyed set union; the CRDT merge operation (W3).
$\sqsubseteq$	Lattice partial order: $F_1 \sqsubseteq F_2$ iff $F_1 \subseteq F_2$ componentwise (monotone growth).

Term / symbol	Meaning
Credence	Per-pick subjective <code>[0,1]</code> confidence carried by its author.
Provenance	PROV-O attribution/derivation/generation metadata on each pick (W5).
Root manifest	CID over the set of strand-head CIDs; summarizes a fabric for cheap diff/reconcile.
Lamport clock	Per-record logical timestamp giving causal (happens-before) order (W12).
Version vector / frontier	Set of per-strand <code>seq</code> heads a peer holds; basis for sync deltas.
PLS / DID	PLS (#1) / Decentralized Identifier (W9) signing identity of a strand author.
Block-lattice (#3)	MeshLattice's value ledger providing total order + objective finality; the weave's complement, not its competitor.
Companion suite	#1 PLS, #2 Context Capsules, #3 MeshLattice, #4 KnitNet, #5 VirtualIPC, #6 Capstone; KnitWeb is #7 (this paper).

A.4 Projection to MeTTa / Hyperon (and validation-by-replay)

A KnitWeb sub-weave projects losslessly to MeTTa as a set of **flat, typed, reified atoms** — one atom per (predicate, subject, object) — so a weft pick and its provenance become S-expressions in an OpenCog Hyperon Atomspace (Goertzel et al., 2023). The projection is **deterministic** (stable content-addressed ids; atoms restricted to `[A-Za-z0-9_./:-]` ; literals JSON-quoted), so two peers projecting the same sub-weave emit byte-identical MeTTa — the projection is itself content-addressable and dedups under the CRDT merge (§6).

```
(Pick pick:bafy...)
(PickWarp pick:bafy... warp:entity:acme)
(PickWarp pick:bafy... warp:topic:emissions)
(PickStatement pick:bafy... "acme reported 1.2 Mt C02e in 2025")
(PickProvenance pick:bafy... did:key:z6Mk...)
(PickCredence pick:bafy... 0.82)
```

Validation-by-replay (§7) maps onto Hyperon directly: an exported sub-weave is validated by loading it into a stock Hyperon container (`trueagi/hyperon:latest`) and, for live experiments, an on-demand `opencog/atomspace:latest` CogServer — any peer recomputes the

same verdict over the same content-addressed bytes, no authority required. This mirrors a deployed reference design (the *RLM* MeTTa/Atomspace bridge in the ClaudeClaw system), where MeTTa is treated as a **deterministic export/bridge, not a canonical store**: the canonical state remains the content-addressed weave.

Appendix B — Heritage, Full Vocabulary, Transport & Governance

B.1 Etymological heritage

knit (OE *cnyttan*, "to knot") and **web** (OE *webb*, "woven fabric") map onto Dutch *breien/weefsel* and onto the body: synapses "knit" memory; Sherrington likened the waking brain to a self-weaving fabric of "flashing shuttles"; skin is a crossed collagen weave that *knits* shut as it heals. Proto-Germanic splits the act (*bregdaną*, braid) from the mass (*bragną*, brain). Chinese fuses them — tissue = *zǔzhī* ("woven & organized like silk"), a network of threads/nerves = *luò* (as in *internet*) — all carrying the silk radical *纟*. KnitWeb is built to that shape: order woven from loose threads by knotting and braiding.

B.2 Precedent: the Textus Receptus (Stephanus, 1550)

Estienne's *Editio Regia* wove 16 manuscripts into one main text plus a **critical margin** of variants, and invented **verse numbering** — the first addressable data-nodes. KnitWeb mechanises the same discipline: each source is a Fiber, the synthesized main text is the Braid, the **margin of variants is the contradiction-tolerant knowledge graph** (§7), and verse-IDs are content-addresses (CIDs). Consensus is derived from thread-tension, not dictated — exactly the credence/CRDT model of §6–§7.

B.3 Full vocabulary crosswalk

Term	Meaning	KnitWeb mapping
Blob	a node's raw, unstructured local mass (unspun fleece)	pre-ingest local store
Spinning	turning Blob mass into ordered threads (replaces "mining")	assertion authoring
Fiber	the SHA-256 thread/token (length = mass = value)	value unit (MeshLattice WARP/WEFT); a signed strand

Term	Meaning	KnitWeb mapping
Knit	interlock two Fibers → a new mutually-signed hash	a transaction / weft-pick contribution
Braid	a node's internal chronological thread-history	per-strand causal order (§6) + lineage
knitweb mesh	local validation rules (tension, mass-balance)	the protocol / SHACL + conservation
Node / Mesh / Net	actor / flat p2p links / rigid IP grid	participants & transport (B.5)
Web	organic semantic growth	the knowledge graph itself
Plexus / Textus	the living 3-D fabric of all knit Fibers	the whole weave (this paper's subject)

B.4 Net vs Web (epistemology)

A *net* is a man-made grid that **captures and holds** (rationalist, syntactic, rigid — blockchain logic). A *web* grows from a centre and **feels and connects** (hermeneutic, semantic, elastic). KnitWeb is deliberately a *web*: a fact has no meaning sitting in a cell, only through the threads run through it.

B.5 Transport (the weave is transport-agnostic)

The weave converges by content-address + CRDT merge regardless of how strands arrive. Strands ride a **Multi-Port Fabric Stack**: Bluetooth-LE Air-Mesh (off-grid, à la BitChat), mDNS `udp/5353` (local discovery), QUIC `4242` / WSS `443` (WAN Knits), and DNS `udp/53` only as a stealth fallback. Discovery uses **Spatial Fibers** (geo-hash) and **Visual Fibers** (on-device YOLO) for AR. Full design: `TECHNICAL_WEAVE_AND_GOVERNANCE.md` §5.

B.6 Enterprise data governance

Because every node/edge is content-addressed and provenance-bearing, the weave is a **federated governance surface with no central catalog**: Communities = brand domains, Business Terms = Warps (+ master-data synonyms), Lineage = the Braid, Policies/DQ-rules = the knitweb weave (SHACL + conservation), Stewardship = reputation-gated authorship, and

issue-management = the Stephanus margin (variants kept with credence, never overwritten).
Mapping: TECHNICAL_WEAVE_AND_GOVERNANCE.md §8 + BUSINESS_GLOSSARY_AND_DATA_MODEL.md .

References

- [1] Nakamoto, 2008, Bitcoin: A Peer-to-Peer Electronic Cash System (blockchain total order, contrast), <https://bitcoin.org/bitcoin.pdf>
 - [2] Baird, 2016, The Swirlds Hashgraph Consensus Algorithm (gossip-about-gossip + virtual voting, contrast), <https://www.swirlds.com/downloads/SWIRLDS-TR-2016-01.pdf>
 - [3] Shapiro, Preguica, Baquero, Zawirski, 2011, Conflict-free Replicated Data Types, <https://inria.hal.science/inria-00609399>
 - [4] Benet, 2014, IPFS / IPLD content-addressing (Merkle DAG), <https://arxiv.org/abs/1407.3561>
 - [5] W3C, PROV-O: The PROV Ontology, <https://www.w3.org/TR/prov-o/>
 - [6] W3C, Shapes Constraint Language (SHACL), <https://www.w3.org/TR/shacl/>
 - [7] W3C, RDF 1.1 Concepts, <https://www.w3.org/TR/rdf11-concepts/>
 - [8] Kamvar et al., 2003, EigenTrust, <https://nlp.stanford.edu/pubs/eigentrust.pdf>
 - [9] W3C, 2022, Decentralized Identifiers (DIDs) v1.0, <https://www.w3.org/TR/did-core/>
 - [10] Secure Scuttlebutt Protocol Guide (append-only feeds), <https://ssbc.github.io/scuttlebutt-protocol-guide/>
 - [11] Nano, Open Representative Voting / block-lattice (contrast), <https://docs.nano.org/protocol-design/orv-consensus/>
 - [12] Lamport, 1978, Time, Clocks, and the Ordering of Events, <https://lamport.azurewebsites.net/pubs/time-clocks.pdf>
 - [13] Goertzel et al., 2023, OpenCog Hyperon (MeTTa), <https://arxiv.org/abs/2310.18318>
 - [14] Companion suite: #1 PLS, #2 Context Capsules, #3 MeshLattice, #4 KnitNet, #5 VirtualPC, #7 Capstone.
-

Disclaimer: technical reference design for a lawful decentralized knowledge commons. Not financial advice.